SHBT261 Mini Project 1 — Experiment Grid Search

Cindy Sun

Systematically trains all model/parameter combinations and saves results.

In [1]:
```
!pip install timm
```

```
Defaulting to user installation because normal site-packages is not writeabl
e
Requirement already satisfied: timm in /shared/home/xis553/.local/lib/python
3.11/site-packages (1.0.25)
Requirement already satisfied: torch in /shared/courseSharedFolders/163602ou
ter/163602/cs1090b-gpu/lib/python3.11/site-packages (from timm) (2.10.0)
Requirement already satisfied: torchvision in /shared/courseSharedFolders/16
3602outer/163602/cs1090b-gpu/lib/python3.11/site-packages (from timm) (0.25.
0)
Requirement already satisfied: pyyaml in /shared/courseSharedFolders/163602o
uter/163602/cs1090b-gpu/lib/python3.11/site-packages (from timm) (6.0.3)
Requirement already satisfied: huggingface_hub in /shared/courseSharedFolder
s/163602outer/163602/cs1090b-gpu/lib/python3.11/site-packages (from timm)
(0.36.0)
Requirement already satisfied: safetensors in /shared/courseSharedFolders/16
3602outer/163602/cs1090b-gpu/lib/python3.11/site-packages (from timm) (0.7.
0)
Requirement already satisfied: filelock in /shared/courseSharedFolders/16360
2outer/163602/cs1090b-gpu/lib/python3.11/site-packages (from huggingface_hub
->timm) (3.20.3)
Requirement already satisfied: fsspec>=2023.5.0 in /shared/courseSharedFolde
rs/163602outer/163602/cs1090b-gpu/lib/python3.11/site-packages (from hugging
face_hub->timm) (2024.3.1)
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /shared/courseSharedF
olders/163602outer/163602/cs1090b-gpu/lib/python3.11/site-packages (from hug
gingface_hub->timm) (1.2.1)
Requirement already satisfied: packaging>=20.9 in /shared/courseSharedFolder
s/163602outer/163602/cs1090b-gpu/lib/python3.11/site-packages (from huggingf
ace_hub->timm) (26.0)
Requirement already satisfied: requests in /shared/courseSharedFolders/16360
2outer/163602/cs1090b-gpu/lib/python3.11/site-packages (from huggingface_hub
->timm) (2.31.0)
Requirement already satisfied: tqdm>=4.42.1 in /shared/courseSharedFolders/1
63602outer/163602/cs1090b-gpu/lib/python3.11/site-packages (from huggingface
_hub->timm) (4.66.1)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /shared/courseS
haredFolders/163602outer/163602/cs1090b-gpu/lib/python3.11/site-packages (fr
om huggingface_hub->timm) (4.15.0)
Requirement already satisfied: sympy>=1.13.3 in /shared/courseSharedFolders/
163602outer/163602/cs1090b-gpu/lib/python3.11/site-packages (from torch->tim
m) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /shared/courseSharedFolder
s/163602outer/163602/cs1090b-gpu/lib/python3.11/site-packages (from torch->t
imm) (3.6.1)
Requirement already satisfied: jinja2 in /shared/courseSharedFolders/163602o
uter/163602/cs1090b-gpu/lib/python3.11/site-packages (from torch->timm) (3.
1.6)
Requirement already satisfied: numpy in /shared/courseSharedFolders/163602ou
ter/163602/cs1090b-gpu/lib/python3.11/site-packages (from torchvision->timm)
(1.26.3)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /shared/courseShared
Folders/163602outer/163602/cs1090b-gpu/lib/python3.11/site-packages (from to
rchvision->timm) (11.0.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /shared/courseSharedFol
ders/163602outer/163602/cs1090b-gpu/lib/python3.11/site-packages (from sympy
>=1.13.3->torch->timm) (1.3.0)
```

```
Requirement already satisfied: MarkupSafe>=2.0 in /shared/courseSharedFolder
s/163602outer/163602/cs1090b-gpu/lib/python3.11/site-packages (from jinja2->
torch->timm) (3.0.3)
Requirement already satisfied: charset-normalizer<4,>=2 in /shared/courseSha
redFolders/163602outer/163602/cs1090b-gpu/lib/python3.11/site-packages (from
requests->huggingface_hub->timm) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /shared/courseSharedFolders/1
63602outer/163602/cs1090b-gpu/lib/python3.11/site-packages (from requests->h
uggingface_hub->timm) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /shared/courseSharedFol
ders/163602outer/163602/cs1090b-gpu/lib/python3.11/site-packages (from reque
sts->huggingface_hub->timm) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /shared/courseSharedFol
ders/163602outer/163602/cs1090b-gpu/lib/python3.11/site-packages (from reque
sts->huggingface_hub->timm) (2026.1.4)
```

In [2]:

```python
# ── Cell 1: Imports ──────────────────────────────────────────────────
import os, time, copy, json, warnings
import numpy as np
import pandas as pd
from pathlib import Path
from itertools import product

from PIL import Image
from skimage.feature import hog
from skimage.color import rgb2gray
from tqdm import tqdm

import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import seaborn as sns

from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.metrics import (accuracy_score, confusion_matrix,
                             classification_report, top_k_accuracy_score)

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from torchvision import models, transforms
import timm
import joblib

warnings.filterwarnings('ignore')

print('All imports OK ✓')
print(f'PyTorch  : {torch.__version__}')
print(f'timm     : {timm.__version__}')
print(f'CUDA     : {torch.cuda.is_available()}')
print(f'MPS      : {torch.backends.mps.is_available()}')
```

```
All imports OK ✓
PyTorch  : 2.10.0
timm     : 1.0.25
CUDA     : True
MPS      : False
```

In [3]:
```
# ── Cell 2: Paths ──────────────────────────────────────────────────
print(os.getcwd())
print(os.path.expanduser("~"))

PROJECT_DIR = Path(os.path.expanduser("~")) / "SHBT261_miniproject1"
DATA_DIR    = PROJECT_DIR / "data" / "caltech-101"
FIG_DIR     = PROJECT_DIR / "figures" / "experiments"
MODEL_DIR   = PROJECT_DIR / "models" / "experiments"
RESULTS_DIR = PROJECT_DIR / "results"

for d in [FIG_DIR, MODEL_DIR, RESULTS_DIR]:
    d.mkdir(parents=True, exist_ok=True)

DEVICE = torch.device("mps"  if torch.backends.mps.is_available()
                 else "cuda" if torch.cuda.is_available()
                 else "cpu")

print(f"Project : {PROJECT_DIR}")
print(f"Data    : {DATA_DIR}")
print(f"Device  : {DEVICE}")
print(f"Data exists: {DATA_DIR.exists()}")
```

```
/shared/home/xis553/SHBT261_miniproject1
/shared/home/xis553
Project : /shared/home/xis553/SHBT261_miniproject1
Data    : /shared/home/xis553/SHBT261_miniproject1/data/caltech-101
Device  : cuda
Data exists: True
```

In [4]:
```
# ── Cell 3: Load Data & Split ──────────────────────────────────────
records = []
for cls_folder in sorted(DATA_DIR.iterdir()):
    if cls_folder.is_dir() and not cls_folder.name.startswith('.'):
        for img_path in cls_folder.glob("*.jpg"):
            records.append({"image_path": str(img_path),
                            "class_name": cls_folder.name})

df = pd.DataFrame(records)
le = LabelEncoder()
df["label"] = le.fit_transform(df["class_name"])
NUM_CLASSES  = df["label"].nunique()

train_df, temp_df = train_test_split(df,      test_size=0.30, stratify=df["l
val_df,   test_df = train_test_split(temp_df, test_size=0.50, stratify=temp_
train_df = train_df.reset_index(drop=True)
val_df   = val_df.reset_index(drop=True)
test_df  = test_df.reset_index(drop=True)

print(f"Total images : {len(df)}")
print(f"Classes      : {NUM_CLASSES}")
```

```
print(f"Train        : {len(train_df)}")
print(f"Val          : {len(val_df)}")
print(f"Test         : {len(test_df)}")
```

```
Total images : 8677
Classes      : 101
Train        : 6073
Val          : 1302
Test         : 1302
```

In [5]:

```
# ── Cell 4: HOG Feature Extraction (SVM only) ──────────────────────────
HOG_CACHE = MODEL_DIR.parent.parent / "models"

def extract_hog(img_path, size):
    img      = Image.open(img_path).convert("RGB").resize(size)
    img_gray = rgb2gray(np.array(img))
    return hog(img_gray, orientations=9, pixels_per_cell=(8,8),
               cells_per_block=(2,2), feature_vector=True)

def build_hog_arrays(dataframe, size):
    X, y = [], []
    for _, row in tqdm(dataframe.iterrows(), total=len(dataframe), leave=Fal
        X.append(extract_hog(row["image_path"], size))
        y.append(row["label"])
    return np.array(X, dtype=np.float32), np.array(y)

def get_hog_arrays(size):
    """Load from cache or extract HOG features for a given image size."""
    tag = f"{size[0]}x{size[1]}"
    cache_files = {
        "X_train": MODEL_DIR / f"hog_{tag}_X_train.npy",
        "y_train": MODEL_DIR / f"hog_{tag}_y_train.npy",
        "X_val":   MODEL_DIR / f"hog_{tag}_X_val.npy",
        "y_val":   MODEL_DIR / f"hog_{tag}_y_val.npy",
        "X_test":  MODEL_DIR / f"hog_{tag}_X_test.npy",
        "y_test":  MODEL_DIR / f"hog_{tag}_y_test.npy",
    }
    if all(f.exists() for f in cache_files.values()):
        print(f"  HOG {tag} cache found — loading...")
        return {k: np.load(v) for k, v in cache_files.items()}

    print(f"  Extracting HOG {tag} features (first time)...")
    X_tr, y_tr = build_hog_arrays(train_df, size)
    X_va, y_va = build_hog_arrays(val_df,   size)
    X_te, y_te = build_hog_arrays(test_df,  size)

    arrays = {"X_train": X_tr, "y_train": y_tr,
              "X_val":   X_va, "y_val":   y_va,
              "X_test":  X_te, "y_test":  y_te}
    for k, v in arrays.items():
        np.save(cache_files[k], v)
    print(f"  HOG {tag} saved ✓")
    return arrays

print("HOG extraction functions ready ✓")
```

```
HOG extraction functions ready ✓
```

In [6]:
```
# ── Cell 5: Dataset & DataLoader Helpers ──────────────────────────────
NUM_WORKERS = 12  # ondemand server
BATCH_SIZE  = 32

class Caltech101Dataset(Dataset):
    def __init__(self, dataframe, transform=None):
        self.df        = dataframe
        self.transform = transform

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        row   = self.df.iloc[idx]
        image = Image.open(row["image_path"]).convert("RGB")
        if self.transform:
            image = self.transform(image)
        return image, torch.tensor(row["label"], dtype=torch.long)


def get_transforms(img_size, augment=True):
    """Returns train and val/test transforms."""
    mean = [0.485, 0.456, 0.406]
    std  = [0.229, 0.224, 0.225]

    if augment:
        train_tf = transforms.Compose([
            transforms.Resize((img_size, img_size)),
            transforms.RandomHorizontalFlip(),
            transforms.RandomRotation(10),
            transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=
            transforms.ToTensor(),
            transforms.Normalize(mean, std)
        ])
    else:
        train_tf = transforms.Compose([
            transforms.Resize((img_size, img_size)),
            transforms.ToTensor(),
            transforms.Normalize(mean, std)
        ])

    val_tf = transforms.Compose([
        transforms.Resize((img_size, img_size)),
        transforms.ToTensor(),
        transforms.Normalize(mean, std)
    ])
    return train_tf, val_tf


def get_loaders(img_size, augment=True):
    """Returns train, val, test DataLoaders."""
    train_tf, val_tf = get_transforms(img_size, augment)
    train_loader = DataLoader(Caltech101Dataset(train_df, train_tf),
                              batch_size=BATCH_SIZE, shuffle=True,
```

```
                                num_workers=NUM_WORKERS, pin_memory=True)
    val_loader   = DataLoader(Caltech101Dataset(val_df,   val_tf),
                                batch_size=BATCH_SIZE, shuffle=False,
                                num_workers=NUM_WORKERS, pin_memory=True)
    test_loader  = DataLoader(Caltech101Dataset(test_df,  val_tf),
                                batch_size=BATCH_SIZE, shuffle=False,
                                num_workers=NUM_WORKERS, pin_memory=True)
    return train_loader, val_loader, test_loader

print("Dataset helpers ready ✓")
```

```
Dataset helpers ready ✓
```

In [7]:

```
# ── Cell 6: Model Builders ──────────────────────────────────────────
def build_model(model_name, num_classes, freeze_backbone=True):
    """Build any supported model by name."""

    if model_name == "resnet18":
        model = models.resnet18(weights="IMAGENET1K_V1")
        if freeze_backbone:
            for p in model.parameters(): p.requires_grad = False
        model.fc = nn.Sequential(
            nn.Dropout(0.3),
            nn.Linear(model.fc.in_features, num_classes)
        )

    elif model_name == "efficientnet_b0":
        model = models.efficientnet_b0(weights="IMAGENET1K_V1")
        if freeze_backbone:
            for p in model.parameters(): p.requires_grad = False
        in_features = model.classifier[1].in_features
        model.classifier = nn.Sequential(
            nn.Dropout(0.3),
            nn.Linear(in_features, num_classes)
        )

    elif model_name == "vit_small":
        model = timm.create_model("vit_small_patch16_224",
                                  pretrained=True, num_classes=0)
        if freeze_backbone:
            for p in model.parameters(): p.requires_grad = False
        model.head = nn.Sequential(
            nn.LayerNorm(model.num_features),
            nn.Dropout(0.3),
            nn.Linear(model.num_features, num_classes)
        )

    else:
        raise ValueError(f"Unknown model: {model_name}")

    return model


def get_optimizer(opt_name, params, lr, weight_decay=1e-4):
    """Return optimizer by name."""
    if opt_name == "adam":
```

```
        return optim.Adam(params, lr=lr)
    elif opt_name == "adamw":
        return optim.AdamW(params, lr=lr, weight_decay=weight_decay)
    elif opt_name == "sgd":
        return optim.SGD(params, lr=lr, momentum=0.9,
                         weight_decay=weight_decay, nesterov=True)
    else:
        raise ValueError(f"Unknown optimizer: {opt_name}")

print("Model builders ready ✓")
```

```
Model builders ready ✓
```

In [8]:

```
# ── Cell 7: Training & Evaluation Functions ──────────────────────────
def train_one_epoch(model, loader, criterion, optimizer, device):
    model.train()
    total_loss, correct = 0.0, 0
    for images, labels in loader:
        images, labels = images.to(device), labels.long().to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss    = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * images.size(0)
        correct    += (outputs.argmax(1) == labels).sum().item()
    n = len(loader.dataset)
    return total_loss / n, correct / n


@torch.no_grad()
def evaluate(model, loader, criterion, device):
    model.eval()
    total_loss, correct = 0.0, 0
    all_preds, all_labels, all_probs = [], [], []
    for images, labels in loader:
        images, labels = images.to(device), labels.long().to(device)
        outputs = model(images)
        loss    = criterion(outputs, labels)
        probs   = torch.softmax(outputs, dim=1)
        preds   = outputs.argmax(1)
        total_loss += loss.item() * images.size(0)
        correct    += (preds == labels).sum().item()
        all_preds.append(preds.cpu().numpy())
        all_labels.append(labels.cpu().numpy())
        all_probs.append(probs.cpu().numpy())
    n = len(loader.dataset)
    return (total_loss / n, correct / n,
            np.concatenate(all_preds),
            np.concatenate(all_labels),
            np.concatenate(all_probs))


def train_dl_model(model, train_loader, val_loader, cfg, device=DEVICE):
    """Full 2-phase training: freeze → train head, unfreeze → fine-tune."""
    history   = {"train_loss": [], "train_acc": [],
```

```
                 "val_loss":   [], "val_acc":   []}
    criterion = nn.CrossEntropyLoss(label_smoothing=0.1)
    best_acc, best_weights = 0.0, copy.deepcopy(model.state_dict())

    phases = [
        {"epochs": cfg["epochs"], "lr": cfg["lr"],      "unfreeze": False},
        {"epochs": cfg["epochs"], "lr": cfg["lr"] / 10, "unfreeze": True},
    ]

    for phase_idx, phase in enumerate(phases):
        if phase["unfreeze"]:
            for p in model.parameters():
                p.requires_grad = True

        trainable = filter(lambda p: p.requires_grad, model.parameters())
        optimizer = get_optimizer(cfg["optimizer"], trainable, phase["lr"])
        scheduler = optim.lr_scheduler.CosineAnnealingLR(
            optimizer, T_max=phase["epochs"]
        )

        desc = "Head only" if not phase["unfreeze"] else "Full fine-tune"
        print(f"  Phase {phase_idx+1} ({desc}) — {phase['epochs']} epochs")

        for epoch in range(phase["epochs"]):
            t0 = time.time()
            tr_loss, tr_acc = train_one_epoch(model, train_loader,
                                              criterion, optimizer, device)
            va_loss, va_acc, _, _, _ = evaluate(model, val_loader,
                                                criterion, device)
            scheduler.step()

            history["train_loss"].append(tr_loss)
            history["train_acc"].append(tr_acc)
            history["val_loss"].append(va_loss)
            history["val_acc"].append(va_acc)

            if va_acc > best_acc:
                best_acc     = va_acc
                best_weights = copy.deepcopy(model.state_dict())

            print(f"    Epoch [{epoch+1:02d}/{phase['epochs']}] "
                  f"Train {tr_acc*100:.1f}% | Val {va_acc*100:.1f}% "
                  f"| {time.time()-t0:.1f}s")

    model.load_state_dict(best_weights)
    return model, history, best_acc


def compute_metrics(y_true, y_pred, y_prob):
    """Compute all required metrics for the report."""
    acc   = accuracy_score(y_true, y_pred)
    top5  = top_k_accuracy_score(y_true, y_prob, k=5)
    cm    = confusion_matrix(y_true, y_pred)
    pca   = cm.diagonal() / cm.sum(axis=1)  # per-class accuracy
    report = classification_report(y_true, y_pred,
                                   target_names=le.classes_,
```

```
                                    digits=3, zero_division=0,
                                    output_dict=True)
    return {
        "accuracy":        acc,
        "top5_accuracy":   top5,
        "macro_f1":        report["macro avg"]["f1-score"],
        "weighted_f1":     report["weighted avg"]["f1-score"],
        "macro_precision": report["macro avg"]["precision"],
        "macro_recall":    report["macro avg"]["recall"],
        "per_class_acc":   pca.tolist(),
        "confusion_matrix": cm.tolist(),
    }

print("Training & evaluation functions ready ✓")
```

```
Training & evaluation functions ready ✓
```

In [9]:

```
# ── Cell 8: Experiment Configuration ──────────────────────────────────
# ═══════════════════════════════════════════════════════════════════════
# Edit this cell to change what gets run
# ═══════════════════════════════════════════════════════════════════════

# SVM configurations (img_size only — other params fixed for practicality)
SVM_CONFIGS = [
    {"model": "svm", "img_size": 128, "C": 10,  "augment": False},
    {"model": "svm", "img_size": 128, "C": 10,  "augment": True},
    {"model": "svm", "img_size": 64,  "C": 10,  "augment": False},
]

# Deep Learning grid — all combinations will be generated
DL_MODELS     = ["resnet18", "efficientnet_b0", "vit_small"]
IMG_SIZES     = [128, 224]
EPOCHS_LIST   = [5, 10]
LR_LIST       = [1e-3, 1e-4]
OPTIMIZERS    = ["adam", "adamw", "sgd"]
AUGMENT_LIST  = [True, False]

# ── Generate all DL combinations ──────────────────────────────────────
DL_CONFIGS = []
for model_name, img_size, epochs, lr, opt, aug in product(
        DL_MODELS, IMG_SIZES, EPOCHS_LIST, LR_LIST, OPTIMIZERS, AUGMENT_LIST

    # ViT requires 224 — skip 128 for ViT
    if model_name == "vit_small" and img_size == 128:
        continue

    DL_CONFIGS.append({
        "model":     model_name,
        "img_size":  img_size,
        "epochs":    epochs,
        "lr":        lr,
        "optimizer": opt,
        "augment":   aug,
    })

print(f"SVM experiments    : {len(SVM_CONFIGS)}")
```

```
print(f"DL experiments     : {len(DL_CONFIGS)}")
print(f"Total experiments  : {len(SVM_CONFIGS) + len(DL_CONFIGS)}")
print()
print("Sample DL configs:")
for c in DL_CONFIGS[:5]:
    print(f"  {c}")
```

```
SVM experiments    : 3
DL experiments     : 120
Total experiments  : 123

Sample DL configs:
  {'model': 'resnet18', 'img_size': 128, 'epochs': 5, 'lr': 0.001, 'optimize
r': 'adam', 'augment': True}
  {'model': 'resnet18', 'img_size': 128, 'epochs': 5, 'lr': 0.001, 'optimize
r': 'adam', 'augment': False}
  {'model': 'resnet18', 'img_size': 128, 'epochs': 5, 'lr': 0.001, 'optimize
r': 'adamw', 'augment': True}
  {'model': 'resnet18', 'img_size': 128, 'epochs': 5, 'lr': 0.001, 'optimize
r': 'adamw', 'augment': False}
  {'model': 'resnet18', 'img_size': 128, 'epochs': 5, 'lr': 0.001, 'optimize
r': 'sgd', 'augment': True}
```

In [10]:

```
# ── Cell 9: Results Tracker ──────────────────────────────────────────
RESULTS_FILE = RESULTS_DIR / "all_results.json"

def load_results():
    if RESULTS_FILE.exists():
        with open(RESULTS_FILE) as f:
            return json.load(f)
    return {}

def save_results(results):
    with open(RESULTS_FILE, "w") as f:
        json.dump(results, f, indent=2)

def make_run_key(cfg):
    """Unique string key for each config."""
    if cfg["model"] == "svm":
        return (f"svm_img{cfg['img_size']}_C{cfg['C']}"
                f"_aug{cfg['augment']}")
    return (f"{cfg['model']}_img{cfg['img_size']}"
            f"_ep{cfg['epochs']}_lr{cfg['lr']}"
            f"_{cfg['optimizer']}_aug{cfg['augment']}")

# Load any existing results (resumes from checkpoint)
all_results = load_results()
print(f"Existing completed runs: {len(all_results)}")
```

```
Existing completed runs: 92
```

In [11]:

```
# ── Cell 10: Run SVM Experiments ─────────────────────────────────────
print("="*60)
print("RUNNING SVM EXPERIMENTS")
print("="*60)

for cfg in SVM_CONFIGS:
```

```
    run_key = make_run_key(cfg)

    if run_key in all_results:
        print(f"  SKIP (already done): {run_key}")
        continue

    print(f"\nRunning: {run_key}")
    t_start = time.time()

    # Get HOG features
    size    = (cfg["img_size"], cfg["img_size"])
    arrays  = get_hog_arrays(size)

    X_tr = arrays["X_train"].copy()
    y_tr = arrays["y_train"]
    X_te = arrays["X_test"].copy()
    y_te = arrays["y_test"]

    # Augment: add flipped versions to train
    if cfg["augment"]:
        print("  Extracting augmented HOG features...")
        X_aug, y_aug = [], []
        for _, row in tqdm(train_df.iterrows(), total=len(train_df), leave=F
            img  = Image.open(row["image_path"]).convert("RGB").resize(size)
            flip = img.transpose(Image.FLIP_LEFT_RIGHT)
            for im in [img, flip]:
                gray = rgb2gray(np.array(im))
                X_aug.append(hog(gray, orientations=9, pixels_per_cell=(8,8)
                                 cells_per_block=(2,2), feature_vector=True)
                y_aug.append(row["label"])
        X_tr = np.array(X_aug, dtype=np.float32)
        y_tr = np.array(y_aug)

    # Scale
    scaler = StandardScaler()
    X_tr_sc = scaler.fit_transform(X_tr)
    X_te_sc = scaler.transform(X_te)

    # Train SVM
    print("  Training SVM (this takes a while)...")
    svm = SVC(kernel="rbf", C=cfg["C"], gamma="scale",
              probability=True, random_state=42)
    svm.fit(X_tr_sc, y_tr)

    # Evaluate
    y_pred = svm.predict(X_te_sc)
    y_prob = svm.predict_proba(X_te_sc)
    metrics = compute_metrics(y_te, y_pred, y_prob)

    # Save model
    joblib.dump(svm,    MODEL_DIR / f"{run_key}_svm.pkl")
    joblib.dump(scaler, MODEL_DIR / f"{run_key}_scaler.pkl")

    # Record results
    elapsed = time.time() - t_start
    all_results[run_key] = {
```

```
        "config":       cfg,
        "metrics":      {k: v for k, v in metrics.items()
                         if k not in ["per_class_acc", "confusion_matrix"]},
        "per_class_acc": metrics["per_class_acc"],
        "confusion_matrix": metrics["confusion_matrix"],
        "train_time_sec": elapsed,
    }
    save_results(all_results)
    print(f"  ✓ Done — Acc: {metrics['accuracy']*100:.2f}% "
          f"| Top-5: {metrics['top5_accuracy']*100:.2f}% "
          f"| Time: {elapsed/60:.1f} min")

print("\nAll SVM experiments complete ✓")
```

```
============================================================
RUNNING SVM EXPERIMENTS
============================================================
  SKIP (already done): svm_img128_C10_augFalse
  SKIP (already done): svm_img128_C10_augTrue
  SKIP (already done): svm_img64_C10_augFalse

All SVM experiments complete ✓
```

In [12]:

```
# ── Cell 11: Run DL Experiments ──────────────────────────────────────
print("="*60)
print("RUNNING DEEP LEARNING EXPERIMENTS")
print(f"Total configs: {len(DL_CONFIGS)}")
print("="*60)

criterion = nn.CrossEntropyLoss(label_smoothing=0.1)

for idx, cfg in enumerate(DL_CONFIGS):
    run_key = make_run_key(cfg)

    if run_key in all_results:
        print(f"[{idx+1}/{len(DL_CONFIGS)}] SKIP: {run_key}")
        continue

    print(f"\n[{idx+1}/{len(DL_CONFIGS)}] Running: {run_key}")
    t_start = time.time()

    try:
        # Build loaders
        train_loader, val_loader, test_loader = get_loaders(
            cfg["img_size"], cfg["augment"]
        )

        # Build model
        model = build_model(cfg["model"], NUM_CLASSES,
                            freeze_backbone=True).to(DEVICE)

        # Train
        model, history, best_val_acc = train_dl_model(
            model, train_loader, val_loader, cfg, DEVICE
        )
```

```
        # Evaluate on test set
        _, _, y_pred, y_true, y_prob = evaluate(
            model, test_loader, criterion, DEVICE
        )
        metrics = compute_metrics(y_true, y_pred, y_prob)

        # Save model weights
        torch.save(model.state_dict(), MODEL_DIR / f"{run_key}.pth")

        # Save training curves figure
        epochs_range = range(1, len(history["train_acc"]) + 1)
        fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
        ax1.plot(epochs_range, [a*100 for a in history["train_acc"]], label=
        ax1.plot(epochs_range, [a*100 for a in history["val_acc"]],   label=
        ax1.axvline(x=cfg["epochs"], color="gray", linestyle="--", label="Ph
        ax1.set_title(f"{run_key}\nAccuracy"); ax1.set_ylabel("Acc (%)"); ax
        ax2.plot(epochs_range, history["train_loss"], label="Train")
        ax2.plot(epochs_range, history["val_loss"],   label="Val")
        ax2.axvline(x=cfg["epochs"], color="gray", linestyle="--")
        ax2.set_title("Loss"); ax2.set_ylabel("Loss"); ax2.legend()
        plt.tight_layout()
        plt.savefig(FIG_DIR / f"{run_key}_curves.png", dpi=100)
        plt.close()

        # Record results
        elapsed = time.time() - t_start
        all_results[run_key] = {
            "config":       cfg,
            "metrics":      {k: v for k, v in metrics.items()
                             if k not in ["per_class_acc", "confusion_matrix
            "per_class_acc":    metrics["per_class_acc"],
            "confusion_matrix": metrics["confusion_matrix"],
            "best_val_acc":  best_val_acc,
            "history":       history,
            "train_time_sec": elapsed,
        }
        save_results(all_results)
        print(f"  ✓ Acc: {metrics['accuracy']*100:.2f}% "
              f"| Top-5: {metrics['top5_accuracy']*100:.2f}% "
              f"| F1: {metrics['macro_f1']:.3f} "
              f"| Time: {elapsed/60:.1f} min")

        # Free GPU memory
        del model
        if torch.cuda.is_available(): torch.cuda.empty_cache()

    except Exception as e:
        print(f"  ✗ FAILED: {e}")
        all_results[run_key] = {"config": cfg, "error": str(e)}
        save_results(all_results)
        continue

print("\nAll DL experiments complete ✓")
```

```
============================================================
RUNNING DEEP LEARNING EXPERIMENTS
Total configs: 120
============================================================
[1/120] SKIP: resnet18_img128_ep5_lr0.001_adam_augTrue
[2/120] SKIP: resnet18_img128_ep5_lr0.001_adam_augFalse
[3/120] SKIP: resnet18_img128_ep5_lr0.001_adamw_augTrue
[4/120] SKIP: resnet18_img128_ep5_lr0.001_adamw_augFalse
[5/120] SKIP: resnet18_img128_ep5_lr0.001_sgd_augTrue
[6/120] SKIP: resnet18_img128_ep5_lr0.001_sgd_augFalse
[7/120] SKIP: resnet18_img128_ep5_lr0.0001_adam_augTrue
[8/120] SKIP: resnet18_img128_ep5_lr0.0001_adam_augFalse
[9/120] SKIP: resnet18_img128_ep5_lr0.0001_adamw_augTrue
[10/120] SKIP: resnet18_img128_ep5_lr0.0001_adamw_augFalse
[11/120] SKIP: resnet18_img128_ep5_lr0.0001_sgd_augTrue
[12/120] SKIP: resnet18_img128_ep5_lr0.0001_sgd_augFalse
[13/120] SKIP: resnet18_img128_ep10_lr0.001_adam_augTrue
[14/120] SKIP: resnet18_img128_ep10_lr0.001_adam_augFalse
[15/120] SKIP: resnet18_img128_ep10_lr0.001_adamw_augTrue
[16/120] SKIP: resnet18_img128_ep10_lr0.001_adamw_augFalse
[17/120] SKIP: resnet18_img128_ep10_lr0.001_sgd_augTrue
[18/120] SKIP: resnet18_img128_ep10_lr0.001_sgd_augFalse
[19/120] SKIP: resnet18_img128_ep10_lr0.0001_adam_augTrue
[20/120] SKIP: resnet18_img128_ep10_lr0.0001_adam_augFalse
[21/120] SKIP: resnet18_img128_ep10_lr0.0001_adamw_augTrue
[22/120] SKIP: resnet18_img128_ep10_lr0.0001_adamw_augFalse
[23/120] SKIP: resnet18_img128_ep10_lr0.0001_sgd_augTrue
[24/120] SKIP: resnet18_img128_ep10_lr0.0001_sgd_augFalse
[25/120] SKIP: resnet18_img224_ep5_lr0.001_adam_augTrue
[26/120] SKIP: resnet18_img224_ep5_lr0.001_adam_augFalse
[27/120] SKIP: resnet18_img224_ep5_lr0.001_adamw_augTrue
[28/120] SKIP: resnet18_img224_ep5_lr0.001_adamw_augFalse
[29/120] SKIP: resnet18_img224_ep5_lr0.001_sgd_augTrue
[30/120] SKIP: resnet18_img224_ep5_lr0.001_sgd_augFalse
[31/120] SKIP: resnet18_img224_ep5_lr0.0001_adam_augTrue
[32/120] SKIP: resnet18_img224_ep5_lr0.0001_adam_augFalse
[33/120] SKIP: resnet18_img224_ep5_lr0.0001_adamw_augTrue
[34/120] SKIP: resnet18_img224_ep5_lr0.0001_adamw_augFalse
[35/120] SKIP: resnet18_img224_ep5_lr0.0001_sgd_augTrue
[36/120] SKIP: resnet18_img224_ep5_lr0.0001_sgd_augFalse
[37/120] SKIP: resnet18_img224_ep10_lr0.001_adam_augTrue
[38/120] SKIP: resnet18_img224_ep10_lr0.001_adam_augFalse
[39/120] SKIP: resnet18_img224_ep10_lr0.001_adamw_augTrue
[40/120] SKIP: resnet18_img224_ep10_lr0.001_adamw_augFalse
[41/120] SKIP: resnet18_img224_ep10_lr0.001_sgd_augTrue
[42/120] SKIP: resnet18_img224_ep10_lr0.001_sgd_augFalse
[43/120] SKIP: resnet18_img224_ep10_lr0.0001_adam_augTrue
[44/120] SKIP: resnet18_img224_ep10_lr0.0001_adam_augFalse
[45/120] SKIP: resnet18_img224_ep10_lr0.0001_adamw_augTrue
[46/120] SKIP: resnet18_img224_ep10_lr0.0001_adamw_augFalse
[47/120] SKIP: resnet18_img224_ep10_lr0.0001_sgd_augTrue
[48/120] SKIP: resnet18_img224_ep10_lr0.0001_sgd_augFalse
[49/120] SKIP: efficientnet_b0_img128_ep5_lr0.001_adam_augTrue
[50/120] SKIP: efficientnet_b0_img128_ep5_lr0.001_adam_augFalse
[51/120] SKIP: efficientnet_b0_img128_ep5_lr0.001_adamw_augTrue
[52/120] SKIP: efficientnet_b0_img128_ep5_lr0.001_adamw_augFalse
```

```
[53/120] SKIP: efficientnet_b0_img128_ep5_lr0.001_sgd_augTrue
[54/120] SKIP: efficientnet_b0_img128_ep5_lr0.001_sgd_augFalse
[55/120] SKIP: efficientnet_b0_img128_ep5_lr0.0001_adam_augTrue
[56/120] SKIP: efficientnet_b0_img128_ep5_lr0.0001_adam_augFalse
[57/120] SKIP: efficientnet_b0_img128_ep5_lr0.0001_adamw_augTrue
[58/120] SKIP: efficientnet_b0_img128_ep5_lr0.0001_adamw_augFalse
[59/120] SKIP: efficientnet_b0_img128_ep5_lr0.0001_sgd_augTrue
[60/120] SKIP: efficientnet_b0_img128_ep5_lr0.0001_sgd_augFalse
[61/120] SKIP: efficientnet_b0_img128_ep10_lr0.001_adam_augTrue
[62/120] SKIP: efficientnet_b0_img128_ep10_lr0.001_adam_augFalse
[63/120] SKIP: efficientnet_b0_img128_ep10_lr0.001_adamw_augTrue
[64/120] SKIP: efficientnet_b0_img128_ep10_lr0.001_adamw_augFalse
[65/120] SKIP: efficientnet_b0_img128_ep10_lr0.001_sgd_augTrue
[66/120] SKIP: efficientnet_b0_img128_ep10_lr0.001_sgd_augFalse
[67/120] SKIP: efficientnet_b0_img128_ep10_lr0.0001_adam_augTrue
[68/120] SKIP: efficientnet_b0_img128_ep10_lr0.0001_adam_augFalse
[69/120] SKIP: efficientnet_b0_img128_ep10_lr0.0001_adamw_augTrue
[70/120] SKIP: efficientnet_b0_img128_ep10_lr0.0001_adamw_augFalse
[71/120] SKIP: efficientnet_b0_img128_ep10_lr0.0001_sgd_augTrue
[72/120] SKIP: efficientnet_b0_img128_ep10_lr0.0001_sgd_augFalse
[73/120] SKIP: efficientnet_b0_img224_ep5_lr0.001_adam_augTrue
[74/120] SKIP: efficientnet_b0_img224_ep5_lr0.001_adam_augFalse
[75/120] SKIP: efficientnet_b0_img224_ep5_lr0.001_adamw_augTrue
[76/120] SKIP: efficientnet_b0_img224_ep5_lr0.001_adamw_augFalse
[77/120] SKIP: efficientnet_b0_img224_ep5_lr0.001_sgd_augTrue
[78/120] SKIP: efficientnet_b0_img224_ep5_lr0.001_sgd_augFalse
[79/120] SKIP: efficientnet_b0_img224_ep5_lr0.0001_adam_augTrue
[80/120] SKIP: efficientnet_b0_img224_ep5_lr0.0001_adam_augFalse
[81/120] SKIP: efficientnet_b0_img224_ep5_lr0.0001_adamw_augTrue
[82/120] SKIP: efficientnet_b0_img224_ep5_lr0.0001_adamw_augFalse
[83/120] SKIP: efficientnet_b0_img224_ep5_lr0.0001_sgd_augTrue
[84/120] SKIP: efficientnet_b0_img224_ep5_lr0.0001_sgd_augFalse
[85/120] SKIP: efficientnet_b0_img224_ep10_lr0.001_adam_augTrue
[86/120] SKIP: efficientnet_b0_img224_ep10_lr0.001_adam_augFalse
[87/120] SKIP: efficientnet_b0_img224_ep10_lr0.001_adamw_augTrue
[88/120] SKIP: efficientnet_b0_img224_ep10_lr0.001_adamw_augFalse
[89/120] SKIP: efficientnet_b0_img224_ep10_lr0.001_sgd_augTrue

[90/120] Running: efficientnet_b0_img224_ep10_lr0.001_sgd_augFalse
  Phase 1 (Head only) — 10 epochs
    Epoch [01/10] Train 25.4% | Val 43.3% | 9.6s
    Epoch [02/10] Train 42.9% | Val 57.8% | 6.7s
    Epoch [03/10] Train 54.3% | Val 66.8% | 6.6s
    Epoch [04/10] Train 62.5% | Val 72.4% | 6.6s
    Epoch [05/10] Train 68.1% | Val 75.3% | 6.6s
    Epoch [06/10] Train 70.7% | Val 78.0% | 6.7s
    Epoch [07/10] Train 72.6% | Val 78.6% | 6.6s
    Epoch [08/10] Train 74.2% | Val 79.0% | 6.7s
    Epoch [09/10] Train 74.7% | Val 79.5% | 6.6s
    Epoch [10/10] Train 74.6% | Val 79.3% | 6.8s
  Phase 2 (Full fine-tune) — 10 epochs
    Epoch [01/10] Train 75.5% | Val 80.5% | 18.3s
    Epoch [02/10] Train 76.7% | Val 80.6% | 18.3s
    Epoch [03/10] Train 76.9% | Val 81.6% | 18.2s
    Epoch [04/10] Train 77.8% | Val 82.0% | 18.2s
    Epoch [05/10] Train 78.6% | Val 82.3% | 18.3s
```

```
    Epoch [06/10] Train 78.6% | Val 82.7% | 18.3s
    Epoch [07/10] Train 79.2% | Val 82.6% | 18.3s
    Epoch [08/10] Train 79.5% | Val 82.9% | 18.3s
    Epoch [09/10] Train 79.8% | Val 82.3% | 18.4s
    Epoch [10/10] Train 80.0% | Val 82.6% | 18.4s
  ✓ Acc: 81.34% | Top-5: 95.55% | F1: 0.672 | Time: 4.3 min

[91/120] Running: efficientnet_b0_img224_ep10_lr0.0001_adam_augTrue
  Phase 1 (Head only) — 10 epochs
    Epoch [01/10] Train 25.5% | Val 47.8% | 6.9s
    Epoch [02/10] Train 44.3% | Val 60.1% | 6.9s
    Epoch [03/10] Train 56.8% | Val 70.4% | 6.8s
    Epoch [04/10] Train 66.0% | Val 75.0% | 6.8s
    Epoch [05/10] Train 72.4% | Val 78.3% | 6.8s
    Epoch [06/10] Train 76.2% | Val 80.2% | 6.8s
    Epoch [07/10] Train 78.7% | Val 80.9% | 6.8s
    Epoch [08/10] Train 80.3% | Val 81.9% | 6.8s
    Epoch [09/10] Train 79.8% | Val 81.9% | 6.7s
    Epoch [10/10] Train 80.9% | Val 81.6% | 6.8s
  Phase 2 (Full fine-tune) — 10 epochs
    Epoch [01/10] Train 82.6% | Val 85.1% | 18.5s
    Epoch [02/10] Train 85.1% | Val 87.8% | 18.6s
    Epoch [03/10] Train 86.8% | Val 88.7% | 18.5s
    Epoch [04/10] Train 88.3% | Val 89.7% | 18.5s
    Epoch [05/10] Train 90.3% | Val 90.6% | 18.6s
    Epoch [06/10] Train 91.0% | Val 91.1% | 18.4s
    Epoch [07/10] Train 91.6% | Val 92.0% | 18.5s
    Epoch [08/10] Train 92.2% | Val 92.4% | 18.6s
    Epoch [09/10] Train 91.8% | Val 92.2% | 18.6s
    Epoch [10/10] Train 92.5% | Val 92.5% | 18.6s
  ✓ Acc: 91.94% | Top-5: 99.16% | F1: 0.875 | Time: 4.3 min

[92/120] Running: efficientnet_b0_img224_ep10_lr0.0001_adam_augFalse
  Phase 1 (Head only) — 10 epochs
    Epoch [01/10] Train 26.9% | Val 50.5% | 6.9s
    Epoch [02/10] Train 49.2% | Val 65.7% | 6.8s
    Epoch [03/10] Train 63.2% | Val 75.4% | 6.8s
    Epoch [04/10] Train 72.2% | Val 80.9% | 6.7s
    Epoch [05/10] Train 78.5% | Val 83.5% | 6.8s
    Epoch [06/10] Train 82.2% | Val 84.3% | 6.7s
    Epoch [07/10] Train 83.2% | Val 85.1% | 6.8s
    Epoch [08/10] Train 86.1% | Val 85.9% | 6.7s
    Epoch [09/10] Train 85.3% | Val 85.6% | 6.7s
    Epoch [10/10] Train 86.0% | Val 85.5% | 6.7s
  Phase 2 (Full fine-tune) — 10 epochs
    Epoch [01/10] Train 86.1% | Val 88.1% | 18.6s
    Epoch [02/10] Train 88.3% | Val 89.6% | 18.6s
    Epoch [03/10] Train 90.2% | Val 91.5% | 18.6s
    Epoch [04/10] Train 92.2% | Val 92.2% | 18.6s
    Epoch [10/10] Train 94.4% | Val 94.0% | 18.6s
  ✓ Acc: 93.86% | Top-5: 99.08% | F1: 0.904 | Time: 4.3 min

[93/120] Running: efficientnet_b0_img224_ep10_lr0.0001_adamw_augTrue
  Phase 1 (Head only) — 10 epochs
    Epoch [01/10] Train 24.6% | Val 48.4% | 7.0s
    Epoch [02/10] Train 44.9% | Val 61.7% | 6.9s
```

```
    Epoch [03/10] Train 56.4% | Val 70.9% | 6.9s
    Epoch [04/10] Train 66.1% | Val 77.0% | 7.0s
    Epoch [05/10] Train 72.9% | Val 79.7% | 7.0s
    Epoch [06/10] Train 75.7% | Val 81.6% | 6.9s
    Epoch [07/10] Train 78.5% | Val 81.6% | 6.9s
    Epoch [08/10] Train 79.9% | Val 82.4% | 6.9s
    Epoch [09/10] Train 80.5% | Val 82.5% | 6.9s
    Epoch [10/10] Train 81.0% | Val 83.0% | 6.9s
  Phase 2 (Full fine-tune) — 10 epochs
    Epoch [01/10] Train 82.3% | Val 85.7% | 18.7s
    Epoch [02/10] Train 85.2% | Val 87.9% | 18.7s
    Epoch [03/10] Train 87.3% | Val 89.2% | 18.8s
    Epoch [04/10] Train 88.6% | Val 90.8% | 18.7s
    Epoch [05/10] Train 89.8% | Val 91.4% | 18.7s
    Epoch [02/10] Train 50.8% | Val 67.8% | 6.8s
    Epoch [03/10] Train 64.4% | Val 75.7% | 6.8s
    Epoch [04/10] Train 73.9% | Val 81.0% | 6.7s
    Epoch [05/10] Train 78.8% | Val 83.3% | 6.8s
    Epoch [06/10] Train 82.4% | Val 84.4% | 6.8s
    Epoch [07/10] Train 84.2% | Val 84.3% | 6.8s
    Epoch [08/10] Train 84.9% | Val 84.6% | 6.8s
    Epoch [09/10] Train 85.5% | Val 85.6% | 6.8s
    Epoch [10/10] Train 86.1% | Val 85.8% | 6.8s
  Phase 2 (Full fine-tune) — 10 epochs
    Epoch [01/10] Train 86.8% | Val 87.6% | 18.6s
    Epoch [02/10] Train 88.6% | Val 89.4% | 18.6s
    Epoch [03/10] Train 90.3% | Val 90.2% | 18.6s
    Epoch [04/10] Train 91.5% | Val 91.3% | 18.6s
    Epoch [05/10] Train 92.4% | Val 92.4% | 18.8s
    Epoch [06/10] Train 93.3% | Val 92.8% | 18.7s
    Epoch [07/10] Train 93.8% | Val 93.6% | 18.7s
    Epoch [08/10] Train 94.3% | Val 93.3% | 18.6s
    Epoch [09/10] Train 94.6% | Val 93.5% | 18.7s
    Epoch [10/10] Train 94.4% | Val 93.4% | 18.6s
  ✓ Acc: 93.86% | Top-5: 99.23% | F1: 0.899 | Time: 4.3 min

[95/120] Running: efficientnet_b0_img224_ep10_lr0.0001_sgd_augTrue
  Phase 1 (Head only) — 10 epochs
    Epoch [01/10] Train 7.0% | Val 20.7% | 7.0s
    Epoch [02/10] Train 22.2% | Val 26.4% | 7.0s
    Epoch [03/10] Train 25.7% | Val 28.5% | 6.9s
    Epoch [04/10] Train 26.3% | Val 29.2% | 6.9s
    Epoch [05/10] Train 27.1% | Val 30.4% | 6.9s
    Epoch [06/10] Train 27.4% | Val 30.6% | 7.0s
    Epoch [07/10] Train 27.9% | Val 31.3% | 6.9s
    Epoch [08/10] Train 27.8% | Val 31.6% | 6.9s
    Epoch [09/10] Train 28.0% | Val 31.9% | 6.9s
    Epoch [10/10] Train 28.5% | Val 32.3% | 7.0s
  Phase 2 (Full fine-tune) — 10 epochs
    Epoch [01/10] Train 28.2% | Val 32.1% | 18.7s
    Epoch [02/10] Train 28.4% | Val 32.1% | 18.7s
    Epoch [03/10] Train 28.8% | Val 32.4% | 18.7s
    Epoch [04/10] Train 29.3% | Val 33.3% | 18.7s
    Epoch [05/10] Train 28.9% | Val 32.8% | 19.0s
    Epoch [06/10] Train 29.2% | Val 33.5% | 18.8s
    Epoch [07/10] Train 29.5% | Val 33.6% | 18.8s
```

```
    Epoch [08/10] Train 29.3% | Val 32.9% | 18.8s
    Epoch [09/10] Train 28.9% | Val 33.0% | 18.7s
    Epoch [10/10] Train 29.1% | Val 33.2% | 18.7s
  ✓ Acc: 33.41% | Top-5: 51.69% | F1: 0.102 | Time: 4.3 min

[96/120] Running: efficientnet_b0_img224_ep10_lr0.0001_sgd_augFalse
  Phase 1 (Head only) — 10 epochs
    Epoch [01/10] Train 5.8% | Val 21.4% | 6.9s
    Epoch [02/10] Train 21.9% | Val 27.0% | 6.8s
    Epoch [03/10] Train 25.7% | Val 28.4% | 6.8s
    Epoch [04/10] Train 26.9% | Val 30.5% | 6.7s
    Epoch [05/10] Train 27.8% | Val 32.2% | 6.9s
    Epoch [06/10] Train 28.8% | Val 33.9% | 6.8s
    Epoch [07/10] Train 29.9% | Val 34.6% | 6.9s
    Epoch [08/10] Train 29.9% | Val 34.9% | 6.8s
    Epoch [09/10] Train 29.9% | Val 36.3% | 6.9s
    Epoch [10/10] Train 29.6% | Val 35.3% | 6.8s
  Phase 2 (Full fine-tune) — 10 epochs
    Epoch [01/10] Train 30.5% | Val 36.3% | 18.7s
    Epoch [06/10] Train 30.6% | Val 37.0% | 18.6s
    Epoch [07/10] Train 31.0% | Val 37.3% | 18.6s
    Epoch [08/10] Train 31.6% | Val 38.2% | 18.6s
    Epoch [09/10] Train 30.9% | Val 38.5% | 18.6s
    Epoch [10/10] Train 31.5% | Val 38.5% | 18.6s
  ✓ Acc: 36.41% | Top-5: 53.84% | F1: 0.126 | Time: 4.3 min

[97/120] Running: vit_small_img224_ep5_lr0.001_adam_augTrue
model.safetensors:   0%|          | 0.00/88.2M [00:00<?, ?B/s]
```

```
  Phase 1 (Head only) — 5 epochs
    Epoch [01/5] Train 67.6% | Val 92.9% | 13.0s
    Epoch [02/5] Train 91.8% | Val 94.5% | 12.8s
    Epoch [03/5] Train 94.6% | Val 95.1% | 12.8s
    Epoch [04/5] Train 95.8% | Val 95.3% | 12.6s
    Epoch [05/5] Train 96.5% | Val 95.5% | 12.3s
  Phase 2 (Full fine-tune) — 5 epochs
    Epoch [01/5] Train 95.9% | Val 96.2% | 30.2s
    Epoch [02/5] Train 98.5% | Val 96.9% | 30.3s
    Epoch [03/5] Train 99.5% | Val 97.3% | 30.6s
    Epoch [04/5] Train 99.9% | Val 97.5% | 31.3s
    Epoch [05/5] Train 99.8% | Val 97.8% | 31.6s
  ✓ Acc: 97.62% | Top-5: 99.85% | F1: 0.964 | Time: 3.8 min

[98/120] Running: vit_small_img224_ep5_lr0.001_adam_augFalse
  Phase 1 (Head only) — 5 epochs
    Epoch [01/5] Train 70.7% | Val 92.5% | 12.6s
    Epoch [02/5] Train 94.5% | Val 94.9% | 12.4s
    Epoch [03/5] Train 97.0% | Val 95.2% | 12.3s
    Epoch [04/5] Train 98.1% | Val 95.5% | 12.3s
    Epoch [05/5] Train 98.1% | Val 95.6% | 12.3s
  Phase 2 (Full fine-tune) — 5 epochs
    Epoch [01/5] Train 96.5% | Val 96.6% | 30.7s
    Epoch [02/5] Train 99.2% | Val 97.7% | 31.4s
    Epoch [03/5] Train 99.9% | Val 97.6% | 31.4s
    Epoch [04/5] Train 100.0% | Val 97.6% | 31.4s
    Epoch [05/5] Train 100.0% | Val 97.5% | 31.4s
  ✓ Acc: 97.31% | Top-5: 99.69% | F1: 0.958 | Time: 3.7 min

[99/120] Running: vit_small_img224_ep5_lr0.001_adamw_augTrue
  Phase 1 (Head only) — 5 epochs
    Epoch [01/5] Train 69.2% | Val 93.0% | 14.2s
    Epoch [02/5] Train 92.6% | Val 95.0% | 12.7s
    Epoch [03/5] Train 94.5% | Val 95.2% | 12.7s
    Epoch [04/5] Train 95.3% | Val 95.9% | 12.6s
    Epoch [05/5] Train 95.9% | Val 95.6% | 12.5s
  Phase 2 (Full fine-tune) — 5 epochs
    Epoch [01/5] Train 95.7% | Val 95.7% | 30.8s
    Epoch [02/5] Train 98.6% | Val 97.2% | 31.3s
    Epoch [03/5] Train 99.5% | Val 97.5% | 31.6s
    Epoch [04/5] Train 99.8% | Val 97.8% | 31.6s
    Epoch [05/5] Train 99.9% | Val 97.7% | 31.7s
  ✓ Acc: 97.77% | Top-5: 99.85% | F1: 0.966 | Time: 3.8 min

[100/120] Running: vit_small_img224_ep5_lr0.001_adamw_augFalse
  Phase 1 (Head only) — 5 epochs
    Epoch [01/5] Train 70.7% | Val 92.6% | 14.3s
    Epoch [02/5] Train 94.6% | Val 94.7% | 12.7s
    Epoch [03/5] Train 96.9% | Val 95.2% | 12.9s
    Epoch [04/5] Train 98.0% | Val 95.5% | 13.3s
    Epoch [05/5] Train 98.3% | Val 95.5% | 12.7s
  Phase 2 (Full fine-tune) — 5 epochs
    Epoch [01/5] Train 96.8% | Val 96.4% | 32.0s
    Epoch [02/5] Train 99.0% | Val 97.5% | 31.4s
    Epoch [03/5] Train 99.9% | Val 97.5% | 31.4s
    Epoch [04/5] Train 100.0% | Val 97.5% | 31.5s
```

```
    Epoch [05/5] Train 100.0% | Val 97.6% | 31.4s
  ✓ Acc: 97.70% | Top-5: 99.77% | F1: 0.966 | Time: 3.8 min

[101/120] Running: vit_small_img224_ep5_lr0.001_sgd_augTrue
  Phase 1 (Head only) — 5 epochs
    Epoch [01/5] Train 34.2% | Val 61.1% | 12.7s
    Epoch [02/5] Train 67.0% | Val 81.3% | 12.7s
    Epoch [03/5] Train 78.7% | Val 85.0% | 13.1s
    Epoch [04/5] Train 82.3% | Val 87.1% | 12.6s
    Epoch [05/5] Train 84.6% | Val 87.8% | 12.7s
  Phase 2 (Full fine-tune) — 5 epochs
    Epoch [01/5] Train 88.1% | Val 91.4% | 31.3s
    Epoch [02/5] Train 90.8% | Val 92.0% | 30.8s
    Epoch [03/5] Train 91.6% | Val 92.5% | 30.9s
    Epoch [04/5] Train 92.2% | Val 92.6% | 31.4s
    Epoch [05/5] Train 92.6% | Val 92.9% | 31.1s
  ✓ Acc: 93.24% | Top-5: 98.85% | F1: 0.869 | Time: 3.7 min

[102/120] Running: vit_small_img224_ep5_lr0.001_sgd_augFalse
  Phase 1 (Head only) — 5 epochs
    Epoch [01/5] Train 38.3% | Val 65.6% | 12.7s
    Epoch [02/5] Train 70.5% | Val 82.3% | 12.8s
    Epoch [03/5] Train 81.8% | Val 86.2% | 12.7s
    Epoch [04/5] Train 85.0% | Val 88.2% | 12.6s
    Epoch [05/5] Train 86.7% | Val 88.5% | 12.6s
  Phase 2 (Full fine-tune) — 5 epochs
    Epoch [01/5] Train 89.2% | Val 91.6% | 31.2s
    Epoch [02/5] Train 91.6% | Val 92.6% | 31.2s
    Epoch [03/5] Train 93.0% | Val 92.8% | 31.2s
    Epoch [04/5] Train 93.4% | Val 93.1% | 30.9s
    Epoch [05/5] Train 93.7% | Val 93.1% | 30.9s
  ✓ Acc: 93.01% | Top-5: 99.23% | F1: 0.865 | Time: 3.7 min

[103/120] Running: vit_small_img224_ep5_lr0.0001_adam_augTrue
  Phase 1 (Head only) — 5 epochs
    Epoch [01/5] Train 16.5% | Val 38.5% | 12.6s
    Epoch [02/5] Train 42.0% | Val 60.4% | 12.4s
    Epoch [03/5] Train 57.5% | Val 70.9% | 12.3s
    Epoch [04/5] Train 64.7% | Val 75.0% | 12.2s
    Epoch [05/5] Train 68.5% | Val 75.9% | 12.2s
  Phase 2 (Full fine-tune) — 5 epochs
    Epoch [01/5] Train 82.4% | Val 92.2% | 30.7s
    Epoch [02/5] Train 92.0% | Val 94.5% | 30.9s
    Epoch [03/5] Train 95.2% | Val 95.7% | 30.6s
    Epoch [04/5] Train 96.0% | Val 96.4% | 31.2s
    Epoch [05/5] Train 96.8% | Val 96.6% | 31.8s
  ✓ Acc: 95.78% | Top-5: 99.62% | F1: 0.925 | Time: 3.7 min

[104/120] Running: vit_small_img224_ep5_lr0.0001_adam_augFalse
  Phase 1 (Head only) — 5 epochs
    Epoch [01/5] Train 15.1% | Val 41.2% | 12.7s
    Epoch [02/5] Train 45.1% | Val 59.8% | 12.7s
    Epoch [03/5] Train 60.2% | Val 71.5% | 12.7s
    Epoch [04/5] Train 68.7% | Val 76.5% | 12.7s
    Epoch [05/5] Train 71.9% | Val 77.7% | 12.6s
  Phase 2 (Full fine-tune) — 5 epochs
```

```
    Epoch [01/5] Train 84.3% | Val 91.4% | 31.4s
    Epoch [02/5] Train 93.4% | Val 94.0% | 30.8s
    Epoch [03/5] Train 96.7% | Val 95.3% | 30.5s
    Epoch [04/5] Train 97.6% | Val 95.5% | 31.0s
    Epoch [05/5] Train 98.3% | Val 95.5% | 31.4s
  ✓ Acc: 95.47% | Top-5: 99.39% | F1: 0.922 | Time: 3.7 min

[105/120] Running: vit_small_img224_ep5_lr0.0001_adamw_augTrue
  Phase 1 (Head only) — 5 epochs
    Epoch [01/5] Train 20.9% | Val 42.2% | 12.7s
    Epoch [02/5] Train 44.8% | Val 58.6% | 12.6s
    Epoch [03/5] Train 57.8% | Val 70.2% | 12.5s
    Epoch [04/5] Train 64.9% | Val 73.7% | 12.6s
    Epoch [05/5] Train 69.2% | Val 74.6% | 12.6s
  Phase 2 (Full fine-tune) — 5 epochs
    Epoch [01/5] Train 82.8% | Val 90.5% | 31.8s
    Epoch [02/5] Train 91.2% | Val 93.8% | 32.3s
    Epoch [03/5] Train 94.5% | Val 94.9% | 31.9s
    Epoch [04/5] Train 96.2% | Val 95.9% | 32.2s
    Epoch [05/5] Train 96.7% | Val 96.2% | 31.5s
  ✓ Acc: 96.24% | Top-5: 99.54% | F1: 0.934 | Time: 3.8 min

[106/120] Running: vit_small_img224_ep5_lr0.0001_adamw_augFalse
  Phase 1 (Head only) — 5 epochs
    Epoch [01/5] Train 18.1% | Val 42.5% | 12.3s
    Epoch [02/5] Train 47.2% | Val 64.1% | 12.2s
    Epoch [03/5] Train 63.4% | Val 73.3% | 12.2s
    Epoch [04/5] Train 70.3% | Val 77.3% | 12.3s
    Epoch [05/5] Train 73.8% | Val 79.0% | 12.5s
  Phase 2 (Full fine-tune) — 5 epochs
    Epoch [01/5] Train 84.5% | Val 92.0% | 31.4s
    Epoch [02/5] Train 93.7% | Val 93.9% | 31.9s
    Epoch [03/5] Train 96.6% | Val 95.2% | 30.9s
    Epoch [04/5] Train 97.5% | Val 95.7% | 31.1s
    Epoch [05/5] Train 98.2% | Val 95.7% | 31.9s
  ✓ Acc: 96.39% | Top-5: 99.39% | F1: 0.932 | Time: 3.7 min

[107/120] Running: vit_small_img224_ep5_lr0.0001_sgd_augTrue
  Phase 1 (Head only) — 5 epochs
    Epoch [01/5] Train 10.4% | Val 23.7% | 13.0s
    Epoch [02/5] Train 24.1% | Val 26.0% | 13.0s
    Epoch [03/5] Train 27.3% | Val 29.4% | 12.9s
    Epoch [04/5] Train 29.2% | Val 31.0% | 13.0s
    Epoch [05/5] Train 30.2% | Val 31.4% | 13.0s
  Phase 2 (Full fine-tune) — 5 epochs
    Epoch [01/5] Train 30.7% | Val 33.2% | 31.9s
    Epoch [02/5] Train 32.1% | Val 34.1% | 31.8s
    Epoch [03/5] Train 32.7% | Val 35.2% | 31.9s
    Epoch [04/5] Train 34.0% | Val 35.6% | 31.6s
    Epoch [05/5] Train 33.7% | Val 35.7% | 31.4s
  ✓ Acc: 35.02% | Top-5: 55.22% | F1: 0.095 | Time: 3.8 min

[108/120] Running: vit_small_img224_ep5_lr0.0001_sgd_augFalse
  Phase 1 (Head only) — 5 epochs
    Epoch [01/5] Train 10.3% | Val 24.2% | 12.9s
    Epoch [02/5] Train 25.4% | Val 28.2% | 13.0s
```

```
    Epoch [03/5] Train 28.9% | Val 31.8% | 12.8s
    Epoch [04/5] Train 30.8% | Val 33.6% | 12.5s
    Epoch [05/5] Train 32.1% | Val 34.3% | 12.5s
  Phase 2 (Full fine-tune) — 5 epochs
    Epoch [01/5] Train 32.4% | Val 36.5% | 30.6s
    Epoch [02/5] Train 34.1% | Val 37.9% | 31.1s
    Epoch [03/5] Train 34.9% | Val 38.8% | 31.8s
    Epoch [04/5] Train 35.5% | Val 39.2% | 31.6s
    Epoch [05/5] Train 36.0% | Val 39.2% | 31.2s
  ✓ Acc: 38.86% | Top-5: 57.99% | F1: 0.125 | Time: 3.7 min

[109/120] Running: vit_small_img224_ep10_lr0.001_adam_augTrue
  Phase 1 (Head only) — 10 epochs
    Epoch [01/10] Train 68.5% | Val 91.9% | 12.9s
    Epoch [02/10] Train 92.2% | Val 93.9% | 13.1s
    Epoch [03/10] Train 94.4% | Val 94.7% | 12.9s
    Epoch [04/10] Train 95.8% | Val 95.5% | 12.8s
    Epoch [05/10] Train 96.3% | Val 95.2% | 12.9s
    Epoch [06/10] Train 96.9% | Val 95.4% | 12.9s
    Epoch [07/10] Train 97.1% | Val 95.5% | 12.9s
    Epoch [08/10] Train 97.8% | Val 95.5% | 13.0s
    Epoch [09/10] Train 97.6% | Val 95.5% | 13.0s
    Epoch [10/10] Train 97.6% | Val 95.5% | 13.0s
  Phase 2 (Full fine-tune) — 10 epochs
    Epoch [01/10] Train 96.9% | Val 96.7% | 32.0s
    Epoch [02/10] Train 98.6% | Val 96.9% | 31.8s
    Epoch [03/10] Train 99.0% | Val 96.8% | 31.4s
    Epoch [04/10] Train 99.6% | Val 97.2% | 32.0s
    Epoch [05/10] Train 99.9% | Val 97.0% | 32.0s
    Epoch [06/10] Train 99.8% | Val 97.8% | 32.0s
    Epoch [07/10] Train 99.9% | Val 97.6% | 31.7s
    Epoch [08/10] Train 100.0% | Val 97.9% | 32.0s
    Epoch [09/10] Train 100.0% | Val 97.9% | 31.6s
    Epoch [10/10] Train 100.0% | Val 98.2% | 31.6s
  ✓ Acc: 98.00% | Top-5: 99.69% | F1: 0.969 | Time: 7.5 min

[110/120] Running: vit_small_img224_ep10_lr0.001_adam_augFalse
  Phase 1 (Head only) — 10 epochs
    Epoch [01/10] Train 70.8% | Val 93.5% | 13.0s
    Epoch [02/10] Train 94.3% | Val 94.9% | 12.8s
    Epoch [03/10] Train 96.9% | Val 95.1% | 12.8s
    Epoch [04/10] Train 97.7% | Val 95.0% | 12.9s
    Epoch [05/10] Train 98.4% | Val 95.1% | 13.0s
    Epoch [06/10] Train 98.8% | Val 95.5% | 12.9s
    Epoch [07/10] Train 99.0% | Val 95.8% | 13.0s
    Epoch [08/10] Train 99.4% | Val 95.5% | 13.0s
    Epoch [09/10] Train 99.3% | Val 95.6% | 12.9s
    Epoch [10/10] Train 99.4% | Val 95.5% | 12.9s
  Phase 2 (Full fine-tune) — 10 epochs
    Epoch [01/10] Train 97.4% | Val 96.3% | 31.7s
    Epoch [02/10] Train 99.5% | Val 97.1% | 31.6s
    Epoch [03/10] Train 99.7% | Val 97.7% | 31.3s
    Epoch [04/10] Train 99.9% | Val 97.8% | 31.7s
    Epoch [05/10] Train 99.9% | Val 97.8% | 31.7s
    Epoch [06/10] Train 100.0% | Val 97.8% | 31.9s
    Epoch [07/10] Train 100.0% | Val 97.8% | 31.8s
```

```
    Epoch [08/10] Train 100.0% | Val 97.9% | 31.9s
    Epoch [09/10] Train 100.0% | Val 97.8% | 31.7s
    Epoch [10/10] Train 100.0% | Val 97.8% | 31.6s
  ✓ Acc: 98.00% | Top-5: 99.92% | F1: 0.969 | Time: 7.5 min

[111/120] Running: vit_small_img224_ep10_lr0.001_adamw_augTrue
  Phase 1 (Head only) — 10 epochs
    Epoch [01/10] Train 68.2% | Val 93.2% | 13.0s
    Epoch [02/10] Train 92.7% | Val 94.4% | 13.0s
    Epoch [03/10] Train 94.2% | Val 95.3% | 13.0s
    Epoch [04/10] Train 95.7% | Val 95.4% | 12.9s
    Epoch [05/10] Train 96.2% | Val 95.9% | 12.9s
    Epoch [06/10] Train 96.8% | Val 95.6% | 12.9s
    Epoch [07/10] Train 97.4% | Val 95.9% | 13.0s
    Epoch [08/10] Train 97.4% | Val 95.7% | 13.0s
    Epoch [09/10] Train 97.3% | Val 95.7% | 12.9s
    Epoch [10/10] Train 97.5% | Val 95.8% | 13.0s
  Phase 2 (Full fine-tune) — 10 epochs
    Epoch [01/10] Train 96.8% | Val 97.3% | 32.1s
    Epoch [02/10] Train 98.9% | Val 96.5% | 31.7s
    Epoch [03/10] Train 99.0% | Val 96.9% | 32.1s
    Epoch [04/10] Train 99.6% | Val 97.4% | 31.7s
    Epoch [05/10] Train 99.7% | Val 97.8% | 31.7s
    Epoch [06/10] Train 99.9% | Val 98.0% | 32.4s
    Epoch [07/10] Train 99.9% | Val 97.5% | 32.2s
    Epoch [08/10] Train 100.0% | Val 97.7% | 32.2s
    Epoch [09/10] Train 99.9% | Val 97.7% | 32.1s
    Epoch [10/10] Train 100.0% | Val 97.7% | 32.0s
  ✓ Acc: 98.00% | Top-5: 99.77% | F1: 0.969 | Time: 7.6 min

[112/120] Running: vit_small_img224_ep10_lr0.001_adamw_augFalse
  Phase 1 (Head only) — 10 epochs
    Epoch [01/10] Train 71.2% | Val 93.2% | 12.9s
    Epoch [02/10] Train 94.4% | Val 95.1% | 12.9s
    Epoch [03/10] Train 96.6% | Val 94.9% | 13.0s
    Epoch [04/10] Train 98.1% | Val 95.3% | 12.7s
    Epoch [05/10] Train 98.6% | Val 95.3% | 12.6s
    Epoch [06/10] Train 98.9% | Val 95.5% | 12.5s
    Epoch [07/10] Train 99.1% | Val 95.3% | 12.5s
    Epoch [08/10] Train 99.1% | Val 95.6% | 12.5s
    Epoch [09/10] Train 99.2% | Val 95.5% | 12.6s
    Epoch [10/10] Train 99.4% | Val 95.5% | 12.6s
  Phase 2 (Full fine-tune) — 10 epochs
    Epoch [01/10] Train 97.4% | Val 96.2% | 31.9s
    Epoch [02/10] Train 99.3% | Val 96.8% | 32.1s
    Epoch [03/10] Train 99.8% | Val 97.1% | 31.7s
    Epoch [04/10] Train 100.0% | Val 97.5% | 32.1s
    Epoch [05/10] Train 99.9% | Val 97.8% | 31.9s
    Epoch [06/10] Train 100.0% | Val 97.5% | 31.9s
    Epoch [07/10] Train 100.0% | Val 97.8% | 31.5s
    Epoch [08/10] Train 100.0% | Val 97.8% | 31.7s
    Epoch [09/10] Train 100.0% | Val 97.7% | 31.6s
    Epoch [10/10] Train 100.0% | Val 97.7% | 31.9s
  ✓ Acc: 98.23% | Top-5: 99.69% | F1: 0.974 | Time: 7.5 min

[113/120] Running: vit_small_img224_ep10_lr0.001_sgd_augTrue
```

```
  Phase 1 (Head only) — 10 epochs
    Epoch [01/10] Train 34.8% | Val 64.3% | 14.8s
    Epoch [02/10] Train 67.8% | Val 82.3% | 13.0s
    Epoch [03/10] Train 80.5% | Val 87.7% | 13.0s
    Epoch [04/10] Train 85.2% | Val 90.3% | 13.0s
    Epoch [05/10] Train 87.9% | Val 91.6% | 13.0s
    Epoch [06/10] Train 88.9% | Val 92.5% | 12.9s
    Epoch [07/10] Train 89.9% | Val 92.5% | 13.0s
    Epoch [08/10] Train 89.9% | Val 92.7% | 13.0s
    Epoch [09/10] Train 90.1% | Val 92.9% | 13.0s
    Epoch [10/10] Train 90.8% | Val 92.9% | 12.9s
  Phase 2 (Full fine-tune) — 10 epochs
    Epoch [01/10] Train 93.1% | Val 94.4% | 31.9s
    Epoch [02/10] Train 94.4% | Val 94.9% | 31.8s
    Epoch [06/10] Train 96.3% | Val 95.5% | 31.3s
    Epoch [07/10] Train 96.4% | Val 95.9% | 31.7s
    Epoch [08/10] Train 96.5% | Val 95.9% | 31.8s
    Epoch [09/10] Train 96.5% | Val 95.9% | 31.4s
    Epoch [10/10] Train 96.5% | Val 95.9% | 31.7s
  ✓ Acc: 96.70% | Top-5: 99.69% | F1: 0.950 | Time: 7.5 min

[114/120] Running: vit_small_img224_ep10_lr0.001_sgd_augFalse
  Phase 1 (Head only) — 10 epochs
    Epoch [01/10] Train 38.0% | Val 68.1% | 12.7s
    Epoch [02/10] Train 71.9% | Val 83.3% | 12.8s
    Epoch [03/10] Train 83.1% | Val 88.6% | 12.8s
    Epoch [04/10] Train 88.1% | Val 90.9% | 12.9s
    Epoch [05/10] Train 90.5% | Val 91.9% | 12.9s
    Epoch [06/10] Train 91.9% | Val 92.5% | 12.8s
    Epoch [07/10] Train 92.4% | Val 92.8% | 12.8s
    Epoch [08/10] Train 92.7% | Val 92.9% | 12.8s
    Epoch [09/10] Train 93.2% | Val 92.9% | 12.9s
    Epoch [10/10] Train 93.3% | Val 92.9% | 12.9s
  Phase 2 (Full fine-tune) — 10 epochs
    Epoch [01/10] Train 94.2% | Val 94.4% | 31.5s
    Epoch [02/10] Train 95.6% | Val 94.9% | 31.4s
    Epoch [03/10] Train 96.4% | Val 95.2% | 31.5s
    Epoch [04/10] Train 96.8% | Val 95.4% | 31.3s
    Epoch [05/10] Train 97.1% | Val 95.5% | 31.5s
    Epoch [06/10] Train 97.4% | Val 95.6% | 31.6s
    Epoch [07/10] Train 97.6% | Val 95.7% | 31.5s
    Epoch [08/10] Train 97.7% | Val 95.7% | 31.4s
    Epoch [09/10] Train 97.9% | Val 95.7% | 31.4s
    Epoch [10/10] Train 97.9% | Val 95.7% | 31.3s
  ✓ Acc: 96.70% | Top-5: 99.46% | F1: 0.943 | Time: 7.4 min

[115/120] Running: vit_small_img224_ep10_lr0.0001_adam_augTrue
  Phase 1 (Head only) — 10 epochs
    Epoch [01/10] Train 18.1% | Val 39.9% | 13.2s
    Epoch [02/10] Train 43.4% | Val 63.3% | 13.0s
    Epoch [03/10] Train 60.3% | Val 73.6% | 12.9s
    Epoch [04/10] Train 71.6% | Val 82.3% | 12.9s
    Epoch [05/10] Train 79.4% | Val 86.0% | 12.9s
    Epoch [06/10] Train 81.5% | Val 87.6% | 13.0s
    Epoch [07/10] Train 84.5% | Val 88.6% | 13.0s
    Epoch [08/10] Train 84.5% | Val 89.3% | 13.0s
```

```
    Epoch [09/10] Train 85.7% | Val 89.3% | 13.0s
    Epoch [10/10] Train 85.8% | Val 89.3% | 13.1s
  Phase 2 (Full fine-tune) — 10 epochs
    Epoch [01/10] Train 90.9% | Val 94.0% | 31.9s
    Epoch [02/10] Train 95.4% | Val 95.7% | 31.8s
    Epoch [03/10] Train 97.3% | Val 96.3% | 32.1s
    Epoch [04/10] Train 98.0% | Val 96.9% | 31.8s
    Epoch [05/10] Train 98.3% | Val 96.9% | 31.9s
    Epoch [06/10] Train 98.8% | Val 97.3% | 31.9s
    Epoch [07/10] Train 98.9% | Val 97.6% | 31.7s
    Epoch [08/10] Train 99.0% | Val 97.5% | 31.9s
    Epoch [09/10] Train 99.3% | Val 97.5% | 31.8s
    Epoch [10/10] Train 99.2% | Val 97.4% | 31.4s
  ✓ Acc: 98.00% | Top-5: 99.92% | F1: 0.968 | Time: 7.5 min

[116/120] Running: vit_small_img224_ep10_lr0.0001_adam_augFalse
  Phase 1 (Head only) — 10 epochs
    Epoch [01/10] Train 20.2% | Val 46.2% | 12.3s
    Epoch [02/10] Train 50.0% | Val 65.9% | 12.2s
    Epoch [03/10] Train 67.2% | Val 78.3% | 12.2s
    Epoch [04/10] Train 78.1% | Val 84.3% | 12.4s
    Epoch [05/10] Train 83.1% | Val 86.3% | 12.5s
    Epoch [06/10] Train 86.5% | Val 87.9% | 12.7s
    Epoch [07/10] Train 87.7% | Val 88.9% | 12.8s
    Epoch [08/10] Train 88.2% | Val 89.5% | 13.0s
    Epoch [09/10] Train 89.0% | Val 89.7% | 13.0s
    Epoch [10/10] Train 89.2% | Val 89.9% | 12.6s
  Phase 2 (Full fine-tune) — 10 epochs
    Epoch [01/10] Train 93.0% | Val 94.7% | 30.2s
    Epoch [02/10] Train 96.7% | Val 95.9% | 30.4s
    Epoch [03/10] Train 98.5% | Val 96.3% | 30.6s
    Epoch [04/10] Train 99.3% | Val 96.8% | 31.6s
    Epoch [05/10] Train 99.4% | Val 96.8% | 32.1s
    Epoch [06/10] Train 99.8% | Val 96.7% | 30.3s
    Epoch [10/10] Train 99.9% | Val 96.9% | 31.4s
  ✓ Acc: 97.39% | Top-5: 99.62% | F1: 0.958 | Time: 7.3 min

[117/120] Running: vit_small_img224_ep10_lr0.0001_adamw_augTrue
  Phase 1 (Head only) — 10 epochs
    Epoch [01/10] Train 16.0% | Val 43.0% | 12.9s
    Epoch [02/10] Train 44.3% | Val 62.3% | 13.0s
    Epoch [03/10] Train 59.3% | Val 74.3% | 12.8s
    Epoch [04/10] Train 70.4% | Val 81.7% | 12.8s
    Epoch [05/10] Train 78.0% | Val 86.3% | 12.8s
    Epoch [06/10] Train 80.8% | Val 87.3% | 12.8s
    Epoch [07/10] Train 83.1% | Val 88.1% | 12.9s
    Epoch [08/10] Train 85.1% | Val 88.5% | 12.9s
    Epoch [09/10] Train 85.0% | Val 88.9% | 12.9s
    Epoch [10/10] Train 85.8% | Val 89.0% | 12.8s
  Phase 2 (Full fine-tune) — 10 epochs
    Epoch [01/10] Train 91.3% | Val 94.4% | 31.6s
    Epoch [02/10] Train 95.2% | Val 95.6% | 31.7s
    Epoch [03/10] Train 96.9% | Val 96.3% | 31.6s
    Epoch [04/10] Train 98.1% | Val 96.9% | 31.7s
    Epoch [05/10] Train 98.5% | Val 96.9% | 31.6s
    Epoch [06/10] Train 98.7% | Val 96.9% | 31.7s
```

```
    Epoch [07/10] Train 98.8% | Val 97.0% | 31.6s
    Epoch [08/10] Train 99.0% | Val 96.9% | 31.5s
    Epoch [09/10] Train 99.1% | Val 97.0% | 31.4s
    Epoch [10/10] Train 99.2% | Val 97.0% | 31.4s
  ✓ Acc: 97.70% | Top-5: 99.69% | F1: 0.964 | Time: 7.5 min

[118/120] Running: vit_small_img224_ep10_lr0.0001_adamw_augFalse
  Phase 1 (Head only) — 10 epochs
    Epoch [01/10] Train 20.6% | Val 40.6% | 12.9s
    Epoch [02/10] Train 46.3% | Val 62.4% | 12.7s
    Epoch [03/10] Train 65.2% | Val 76.8% | 12.8s
    Epoch [04/10] Train 76.7% | Val 83.7% | 13.1s
    Epoch [05/10] Train 82.1% | Val 87.5% | 12.7s
    Epoch [06/10] Train 85.5% | Val 88.6% | 12.7s
    Epoch [07/10] Train 87.1% | Val 89.6% | 12.8s
    Epoch [08/10] Train 88.5% | Val 90.2% | 12.8s
    Epoch [09/10] Train 88.2% | Val 90.5% | 12.7s
    Epoch [10/10] Train 89.3% | Val 90.6% | 12.7s
  Phase 2 (Full fine-tune) — 10 epochs
    Epoch [01/10] Train 93.0% | Val 95.0% | 31.5s
    Epoch [02/10] Train 97.3% | Val 95.9% | 31.5s
    Epoch [03/10] Train 98.4% | Val 96.6% | 31.6s
    Epoch [04/10] Train 99.1% | Val 97.0% | 31.7s
    Epoch [05/10] Train 99.5% | Val 97.2% | 31.7s
    Epoch [06/10] Train 99.7% | Val 97.0% | 31.3s
    Epoch [07/10] Train 99.8% | Val 96.9% | 31.5s
    Epoch [08/10] Train 99.9% | Val 97.0% | 31.4s
    Epoch [09/10] Train 99.8% | Val 96.9% | 31.4s
    Epoch [10/10] Train 99.9% | Val 96.9% | 31.2s
  ✓ Acc: 97.62% | Top-5: 99.46% | F1: 0.962 | Time: 7.4 min

[119/120] Running: vit_small_img224_ep10_lr0.0001_sgd_augTrue
  Phase 1 (Head only) — 10 epochs
    Epoch [01/10] Train 13.4% | Val 25.0% | 13.2s
    Epoch [02/10] Train 25.4% | Val 29.6% | 12.8s
    Epoch [03/10] Train 29.5% | Val 34.5% | 12.9s
    Epoch [04/10] Train 33.2% | Val 37.8% | 12.8s
    Epoch [05/10] Train 36.0% | Val 41.0% | 12.8s
    Epoch [06/10] Train 38.6% | Val 44.0% | 12.8s
    Epoch [07/10] Train 39.7% | Val 45.7% | 12.8s
    Epoch [08/10] Train 40.9% | Val 46.9% | 12.8s
    Epoch [09/10] Train 41.6% | Val 47.5% | 12.9s
    Epoch [10/10] Train 42.3% | Val 47.7% | 12.8s
  Phase 2 (Full fine-tune) — 10 epochs
    Epoch [04/10] Train 47.1% | Val 51.5% | 31.1s
    Epoch [05/10] Train 48.4% | Val 51.7% | 31.4s
    Epoch [06/10] Train 48.5% | Val 52.5% | 31.4s
    Epoch [07/10] Train 48.8% | Val 52.8% | 31.5s
    Epoch [08/10] Train 49.3% | Val 52.9% | 31.2s
    Epoch [09/10] Train 49.6% | Val 53.0% | 31.3s
    Epoch [10/10] Train 49.9% | Val 53.0% | 31.3s
  ✓ Acc: 52.15% | Top-5: 74.12% | F1: 0.248 | Time: 7.4 min

[120/120] Running: vit_small_img224_ep10_lr0.0001_sgd_augFalse
  Phase 1 (Head only) — 10 epochs
    Epoch [01/10] Train 11.7% | Val 25.0% | 12.8s
```

```
  Epoch [02/10] Train 25.2% | Val 29.0% | 12.7s
  Epoch [03/10] Train 29.8% | Val 34.7% | 12.7s
  Epoch [04/10] Train 34.4% | Val 40.2% | 12.8s
  Epoch [05/10] Train 38.4% | Val 44.3% | 12.8s
  Epoch [06/10] Train 41.1% | Val 46.9% | 12.5s
  Epoch [07/10] Train 43.6% | Val 48.7% | 12.1s
  Epoch [08/10] Train 44.7% | Val 49.5% | 12.1s
  Epoch [09/10] Train 45.5% | Val 50.2% | 12.1s
  Epoch [10/10] Train 45.7% | Val 50.3% | 12.0s
Phase 2 (Full fine-tune) — 10 epochs
  Epoch [01/10] Train 46.2% | Val 52.2% | 29.8s
  Epoch [02/10] Train 48.3% | Val 53.8% | 30.1s
  Epoch [03/10] Train 49.0% | Val 55.3% | 30.3s
  Epoch [04/10] Train 50.6% | Val 56.0% | 30.1s
  Epoch [05/10] Train 51.9% | Val 57.8% | 30.9s
  Epoch [06/10] Train 52.0% | Val 58.0% | 31.3s
  Epoch [07/10] Train 52.3% | Val 58.3% | 31.2s
  Epoch [08/10] Train 53.2% | Val 58.6% | 31.1s
  Epoch [09/10] Train 53.9% | Val 58.8% | 31.2s
  Epoch [10/10] Train 53.5% | Val 58.9% | 31.1s
✓ Acc: 58.29% | Top-5: 77.73% | F1: 0.323 | Time: 7.3 min

All DL experiments complete ✓
```

In [13]:

```python
# ── Cell 12: Results Summary Table ──────────────────────────────────────
all_results = load_results()  # reload in case you re-run this cell

rows = []
for key, result in all_results.items():
    if "error" in result:
        continue
    cfg = result["config"]
    m   = result["metrics"]
    rows.append({
        "run_key":        key,
        "model":          cfg["model"],
        "img_size":       cfg["img_size"],
        "epochs":         cfg.get("epochs", "N/A"),
        "lr":             cfg.get("lr", "N/A"),
        "optimizer":      cfg.get("optimizer", "rbf"),
        "augment":        cfg["augment"],
        "test_acc_%":     round(m["accuracy"]        * 100, 2),
        "top5_acc_%":     round(m["top5_accuracy"]   * 100, 2),
        "macro_f1":       round(m["macro_f1"],            3),
        "weighted_f1":    round(m["weighted_f1"],         3),
        "macro_precision":round(m["macro_precision"],     3),
        "macro_recall":   round(m["macro_recall"],        3),
        "train_min":      round(result["train_time_sec"] / 60, 1),
    })

results_df = pd.DataFrame(rows).sort_values("test_acc_%", ascending=False)
results_df.to_csv(RESULTS_DIR / "summary_table.csv", index=False)

print(f"Total completed runs: {len(results_df)}")
print("\nTop 10 configurations by test accuracy:")
display(results_df.head(10))
```

```
Total completed runs: 123

Top 10 configurations by test accuracy:
```

	run_key	model	img_size	epochs	lr
114	vit_small_img224_ep10_lr0.001_adamw_augFalse	vit_small	224	10	0.001
117	vit_small_img224_ep10_lr0.0001_adam_augTrue	vit_small	224	10	0.0001
113	vit_small_img224_ep10_lr0.001_adamw_augTrue	vit_small	224	10	0.001
111	vit_small_img224_ep10_lr0.001_adam_augTrue	vit_small	224	10	0.001
112	vit_small_img224_ep10_lr0.001_adam_augFalse	vit_small	224	10	0.001
101	vit_small_img224_ep5_lr0.001_adamw_augTrue	vit_small	224	5	0.001
119	vit_small_img224_ep10_lr0.0001_adamw_augTrue	vit_small	224	10	0.0001
102	vit_small_img224_ep5_lr0.001_adamw_augFalse	vit_small	224	5	0.001
99	vit_small_img224_ep5_lr0.001_adam_augTrue	vit_small	224	5	0.001
120	vit_small_img224_ep10_lr0.0001_adamw_augFalse	vit_small	224	10	0.0001

In [14]:

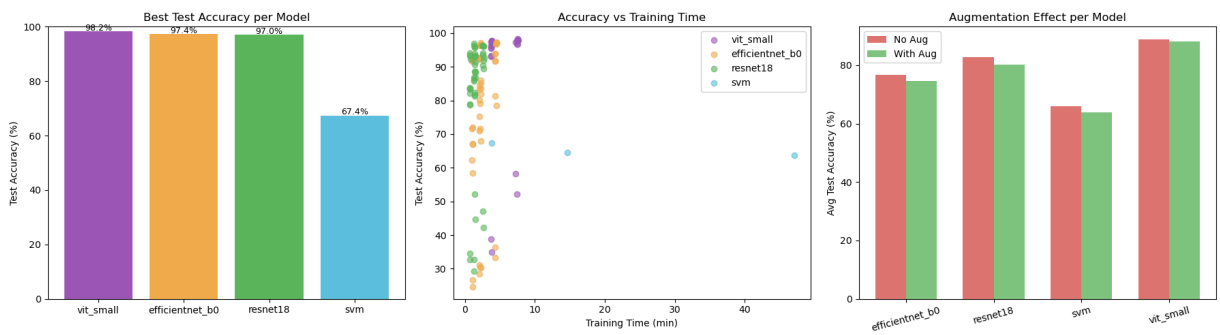
```python
# ── Cell 13: Best Model Per Architecture ──────────────────────────────
print("Best configuration per model:\n")
for model_name in ["svm", "resnet18", "efficientnet_b0", "vit_small"]:
    subset = results_df[results_df["model"] == model_name]
    if subset.empty:
        print(f"  {model_name}: no results yet")
        continue
    best = subset.iloc[0]
    print(f"  {model_name}:")
    print(f"    Accuracy  : {best['test_acc_%']}%")
    print(f"    Top-5     : {best['top5_acc_%']}%")
    print(f"    Macro F1  : {best['macro_f1']}")
    print(f"    Config    : img={best['img_size']}, "
          f"epochs={best['epochs']}, lr={best['lr']}, "
          f"opt={best['optimizer']}, aug={best['augment']}")
    print(f"    Train time: {best['train_min']} min\n")
```

```
Best configuration per model:

  svm:
    Accuracy  : 67.36%
    Top-5     : 84.18%
    Macro F1  : 0.531
    Config    : img=64, epochs=N/A, lr=N/A, opt=rbf, aug=False
    Train time: 3.8 min

  resnet18:
    Accuracy  : 97.0%
    Top-5     : 99.77%
    Macro F1  : 0.954
    Config    : img=224, epochs=5, lr=0.001, opt=adamw, aug=False
    Train time: 1.3 min

  efficientnet_b0:
    Accuracy  : 97.39%
    Top-5     : 99.85%
    Macro F1  : 0.962
    Config    : img=224, epochs=10, lr=0.001, opt=adam, aug=True
    Train time: 4.5 min

  vit_small:
    Accuracy  : 98.23%
    Top-5     : 99.69%
    Macro F1  : 0.974
    Config    : img=224, epochs=10, lr=0.001, opt=adamw, aug=False
    Train time: 7.5 min
```

In [15]:

```
# ── Cell 14: Comparison Bar Chart ──────────────────────────────────────
# Best accuracy per model
best_per_model = results_df.groupby("model")["test_acc_%"].max().reset_index
best_per_model = best_per_model.sort_values("test_acc_%", ascending=False)

colors = {"svm": "#5bc0de", "resnet18": "#5cb85c",
          "efficientnet_b0": "#f0ad4e", "vit_small": "#9b59b6"}

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Plot 1: Best accuracy per model
ax = axes[0]
bars = ax.bar(best_per_model["model"],
              best_per_model["test_acc_%"],
              color=[colors.get(m, "gray") for m in best_per_model["model"]]
for bar, val in zip(bars, best_per_model["test_acc_%"]):
    ax.text(bar.get_x() + bar.get_width()/2, val + 0.3,
            f"{val:.1f}%", ha="center", fontsize=9)
ax.set_title("Best Test Accuracy per Model")
ax.set_ylabel("Test Accuracy (%)")
ax.set_ylim(0, 100)

# Plot 2: Accuracy vs Training Time (scatter)
ax = axes[1]
for model_name in results_df["model"].unique():
```

```
    sub = results_df[results_df["model"] == model_name]
    ax.scatter(sub["train_min"], sub["test_acc_%"],
               label=model_name, alpha=0.6,
               color=colors.get(model_name, "gray"))
ax.set_xlabel("Training Time (min)")
ax.set_ylabel("Test Accuracy (%)")
ax.set_title("Accuracy vs Training Time")
ax.legend()

# Plot 3: Augmentation effect
ax = axes[2]
aug_effect = results_df.groupby(["model", "augment"])["test_acc_%"].mean().u
if True in aug_effect.columns and False in aug_effect.columns:
    x = np.arange(len(aug_effect))
    w = 0.35
    ax.bar(x - w/2, aug_effect[False], width=w, label="No Aug",  color="#d95
    ax.bar(x + w/2, aug_effect[True],  width=w, label="With Aug", color="#5c
    ax.set_xticks(x)
    ax.set_xticklabels(aug_effect.index, rotation=15)
    ax.set_ylabel("Avg Test Accuracy (%)")
    ax.set_title("Augmentation Effect per Model")
    ax.legend()

plt.tight_layout()
plt.savefig(FIG_DIR / "comparison_summary.png", dpi=150)
plt.show()
print("Saved: comparison_summary.png ✓")
```

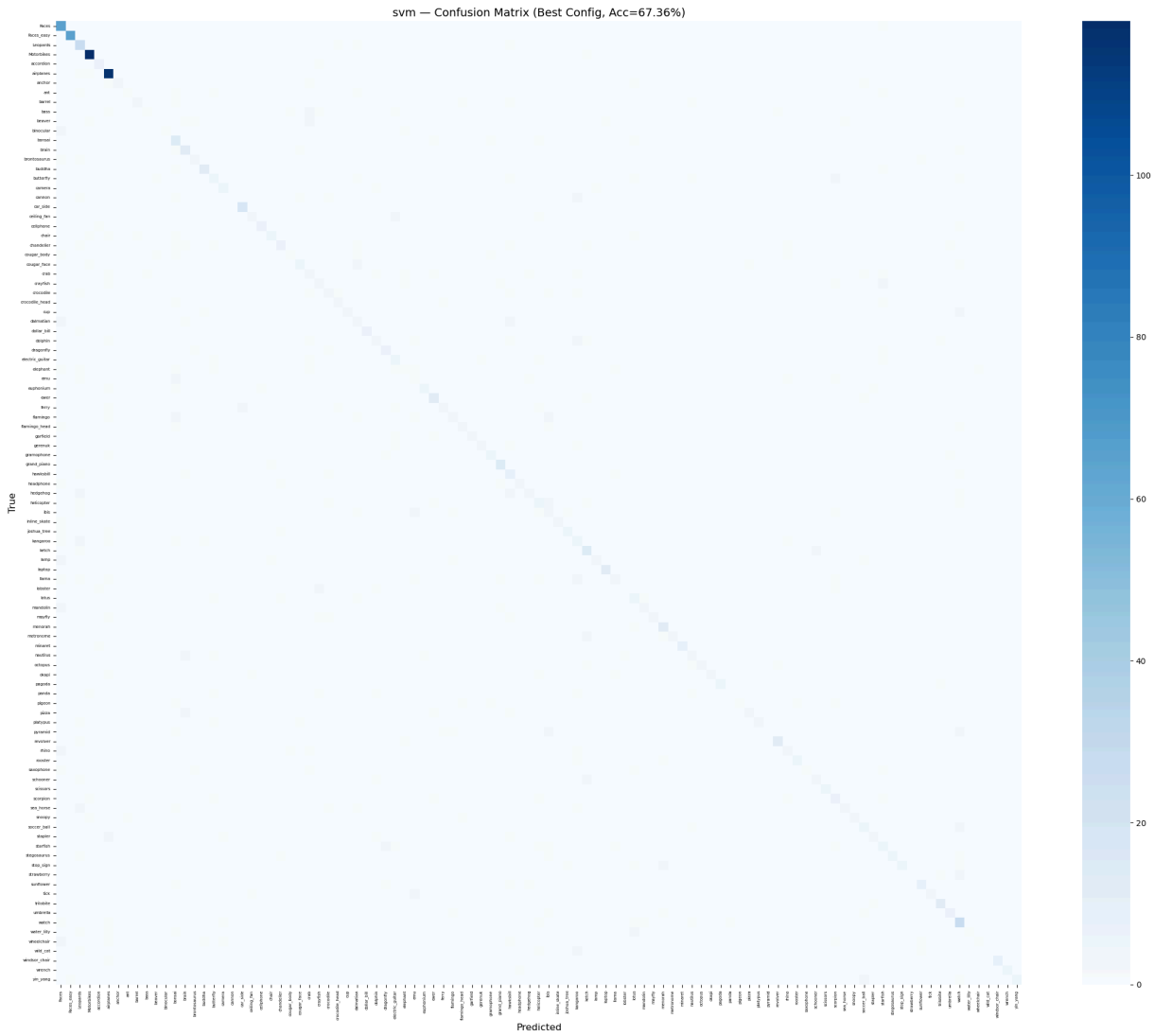



```
Saved: comparison_summary.png ✓
```

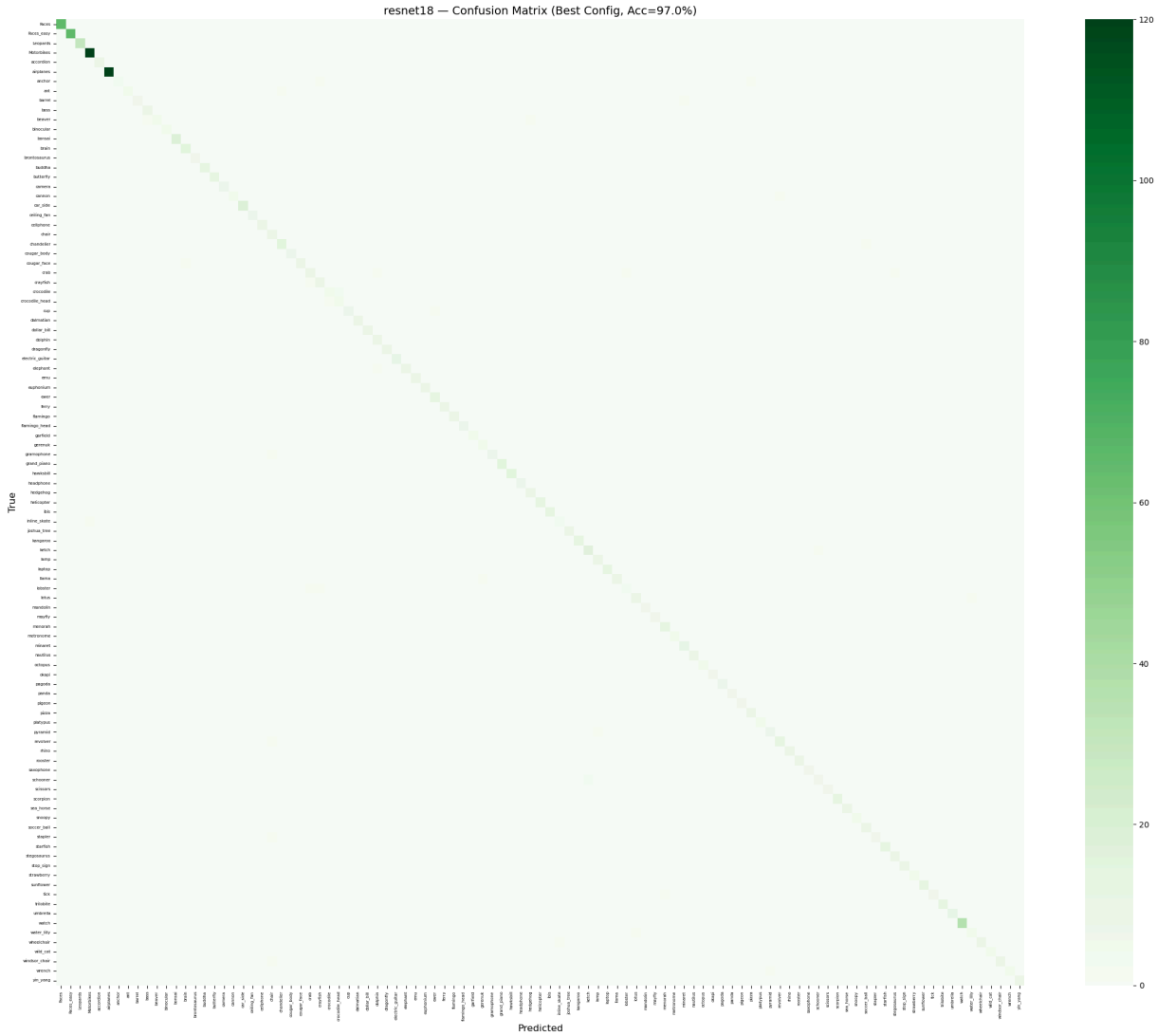
In [16]:

```
# ── Cell 15: Confusion Matrix for Best Config of Each Model ──────────────
for model_name in ["svm", "resnet18", "efficientnet_b0", "vit_small"]:
    subset = results_df[results_df["model"] == model_name]
    if subset.empty:
        continue

    best_key = subset.iloc[0]["run_key"]
    cm_data  = np.array(all_results[best_key]["confusion_matrix"])

    fig, ax = plt.subplots(figsize=(22, 18))
    cmap = {"svm": "Blues", "resnet18": "Greens",
            "efficientnet_b0": "Oranges", "vit_small": "Purples"}
    sns.heatmap(cm_data, annot=False, fmt="d",
                cmap=cmap.get(model_name, "Blues"),
                xticklabels=le.classes_,
                yticklabels=le.classes_, ax=ax)
```

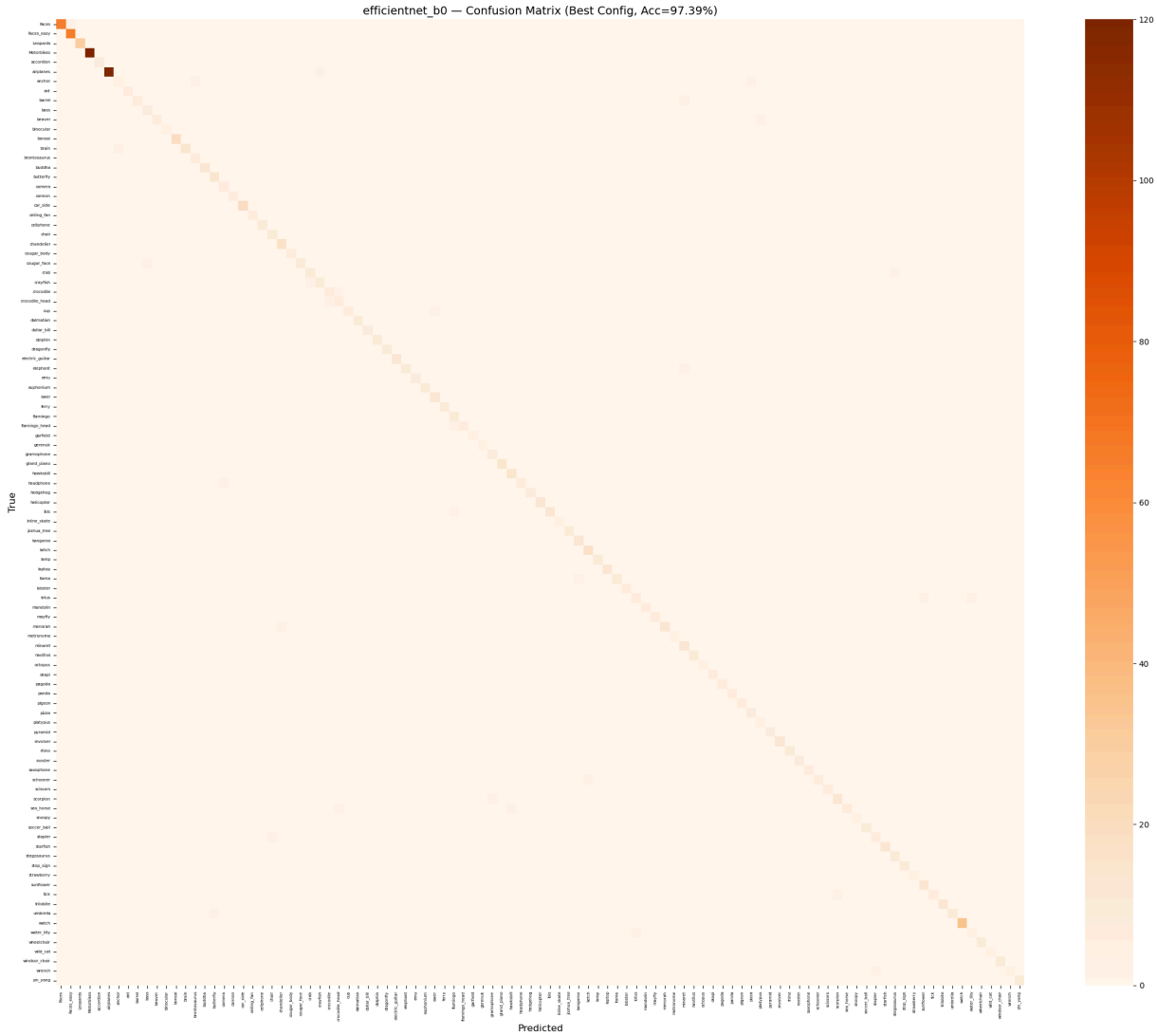
```
    ax.set_xlabel("Predicted", fontsize=12)
    ax.set_ylabel("True",      fontsize=12)
    acc = subset.iloc[0]["test_acc_%"]
    ax.set_title(f"{model_name} — Confusion Matrix (Best Config, Acc={acc}%)
                 fontsize=14)
    plt.xticks(rotation=90, fontsize=5)
    plt.yticks(rotation=0,  fontsize=5)
    plt.tight_layout()
    plt.savefig(FIG_DIR / f"{model_name}_best_confusion_matrix.png", dpi=150
    plt.show()
    print(f"Saved confusion matrix for {model_name} ✓")
```



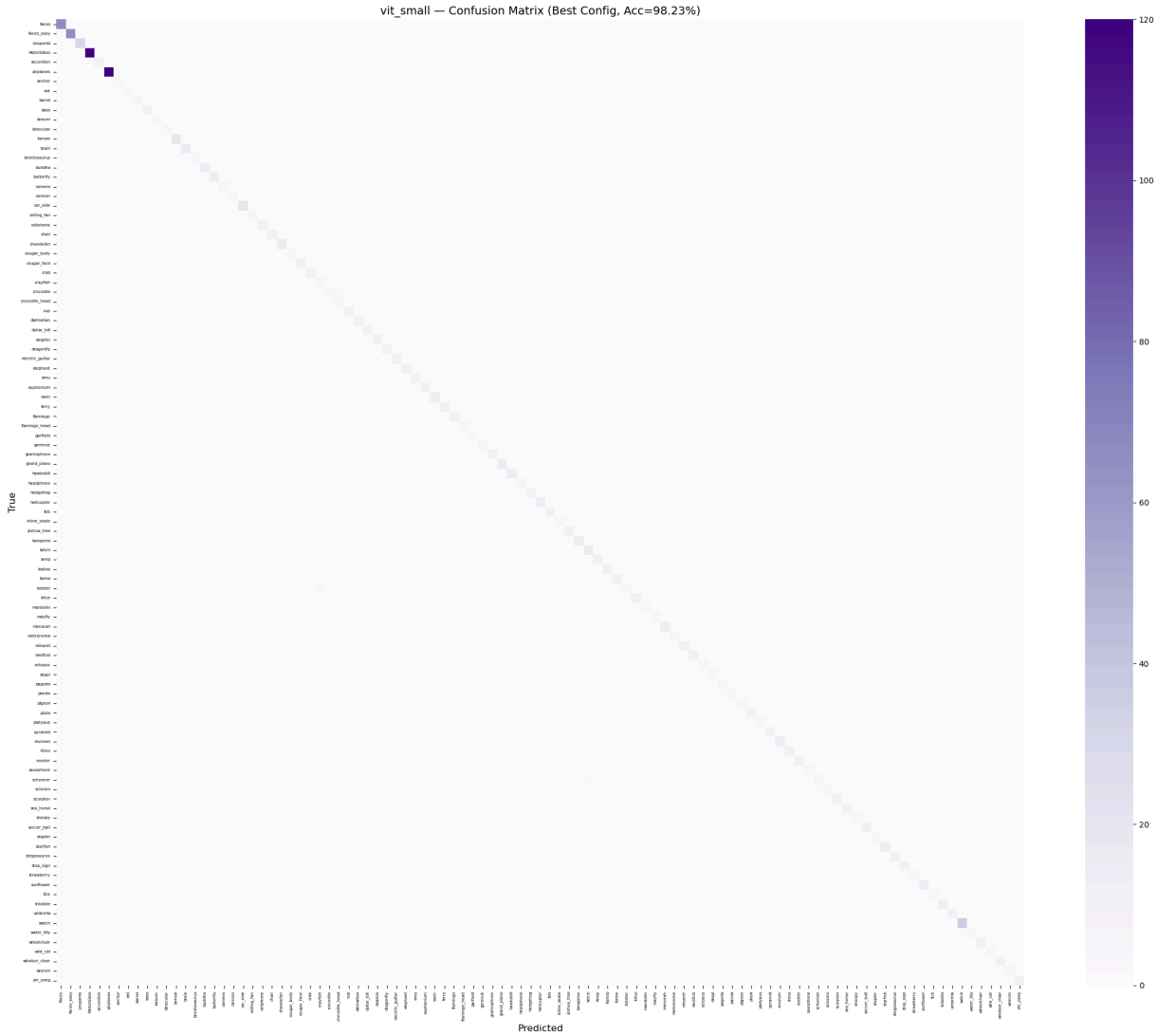

```
Saved confusion matrix for svm ✓
```




```
Saved confusion matrix for resnet18 ✓
```




```
Saved confusion matrix for efficientnet_b0 ✓
```




```
Saved confusion matrix for vit_small ✓
```

In [17]:

```
# ── Cell 16: Per-class Accuracy for Best Config of Each Model ──────────
fig, axes = plt.subplots(2, 2, figsize=(24, 12))
axes = axes.flatten()

color_map = {"svm": "#5bc0de", "resnet18": "#5cb85c",
             "efficientnet_b0": "#f0ad4e", "vit_small": "#9b59b6"}

for i, model_name in enumerate(["svm", "resnet18", "efficientnet_b0", "vit_s
    subset = results_df[results_df["model"] == model_name]
    if subset.empty:
        axes[i].set_title(f"{model_name} — no results")
        continue

    best_key     = subset.iloc[0]["run_key"]
    pca          = np.array(all_results[best_key]["per_class_acc"])
    overall_acc  = subset.iloc[0]["test_acc_%"] / 100

    sorted_idx   = np.argsort(pca)[::-1]
    sorted_pca   = pca[sorted_idx]
    sorted_names = le.classes_[sorted_idx]

    color = color_map.get(model_name, "gray")
```

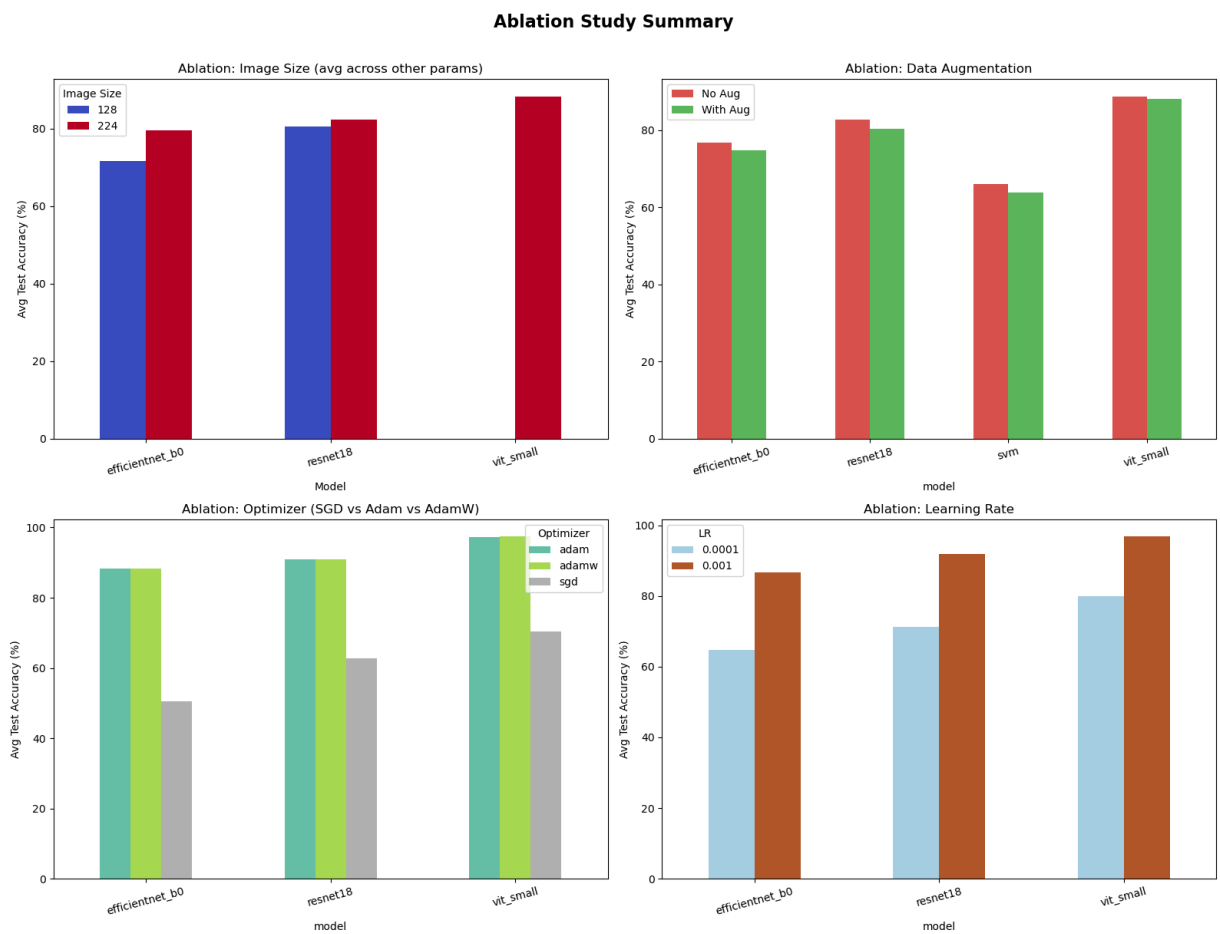
```
    bar_colors = ["#d9534f" if v < 0.5 else color for v in sorted_pca]

    axes[i].bar(range(len(sorted_pca)), sorted_pca, color=bar_colors)
    axes[i].axhline(overall_acc, color="navy", linestyle="--",
                    label=f"Overall ({overall_acc*100:.1f}%)")
    axes[i].set_title(f"{model_name} — Per-class Accuracy (Best Config)")
    axes[i].set_ylabel("Accuracy")
    axes[i].set_xticks(range(len(sorted_names)))
    axes[i].set_xticklabels(sorted_names, rotation=90, fontsize=4)
    axes[i].legend(fontsize=8)

plt.tight_layout()
plt.savefig(FIG_DIR / "all_models_per_class_accuracy.png", dpi=150)
plt.show()
print("Saved: all_models_per_class_accuracy.png ✓")
```

```
Saved: all_models_per_class_accuracy.png ✓
```

In [18]:

```
# ── Cell 17: Ablation Summary Plots ──────────────────────────────
fig, axes = plt.subplots(2, 2, figsize=(16, 12))

# ── Ablation 1: Image Size ───────────────────────────────────────
ax = axes[0, 0]
img_abl = results_df[results_df["model"] != "svm"].groupby(
    ["model", "img_size"])["test_acc_%"].mean().unstack()
img_abl.plot(kind="bar", ax=ax, colormap="coolwarm")
ax.set_title("Ablation: Image Size (avg across other params)")
ax.set_ylabel("Avg Test Accuracy (%)")
ax.set_xlabel("Model")
ax.tick_params(axis='x', rotation=15)
ax.legend(title="Image Size")

# ── Ablation 2: Augmentation ─────────────────────────────────────
ax = axes[0, 1]
aug_abl = results_df.groupby(["model", "augment"])["test_acc_%"].mean().unst
aug_abl.columns = ["No Aug", "With Aug"]
aug_abl.plot(kind="bar", ax=ax, color=["#d9534f", "#5cb85c"])
ax.set_title("Ablation: Data Augmentation")
```

```
ax.set_ylabel("Avg Test Accuracy (%)")
ax.tick_params(axis='x', rotation=15)
ax.legend()

# ── Ablation 3: Optimizer ──────────────────────────────────────────
ax = axes[1, 0]
dl_only = results_df[results_df["model"] != "svm"]
opt_abl = dl_only.groupby(["model", "optimizer"])["test_acc_%"].mean().unsta
opt_abl.plot(kind="bar", ax=ax, colormap="Set2")
ax.set_title("Ablation: Optimizer (SGD vs Adam vs AdamW)")
ax.set_ylabel("Avg Test Accuracy (%)")
ax.tick_params(axis='x', rotation=15)
ax.legend(title="Optimizer")

# ── Ablation 4: Learning Rate ──────────────────────────────────────
ax = axes[1, 1]
lr_abl = dl_only.groupby(["model", "lr"])["test_acc_%"].mean().unstack()
lr_abl.plot(kind="bar", ax=ax, colormap="Paired")
ax.set_title("Ablation: Learning Rate")
ax.set_ylabel("Avg Test Accuracy (%)")
ax.tick_params(axis='x', rotation=15)
ax.legend(title="LR")

plt.suptitle("Ablation Study Summary", fontsize=16, fontweight="bold", y=1.0
plt.tight_layout()
plt.savefig(FIG_DIR / "ablation_summary.png", dpi=150, bbox_inches="tight")
plt.show()
print("Saved: ablation_summary.png ✓")
```



```
Saved: ablation_summary.png ✓
```

In [19]:
```python
# ── Cell 18: Export Final Summary CSV ──────────────────────────────────
results_df.to_csv(RESULTS_DIR / "summary_table.csv", index=False)

print("Files saved to RESULTS_DIR:")
for f in sorted(RESULTS_DIR.iterdir()):
    print(f"  {f.name}")

print("\nFiles saved to FIG_DIR:")
for f in sorted(FIG_DIR.iterdir()):
    print(f"  {f.name}")

print("\nFull experiment grid complete ✓")
print(f"Best overall model: {results_df.iloc[0]['run_key']}")
print(f"Best accuracy     : {results_df.iloc[0]['test_acc_%']}%")
```

```
Files saved to RESULTS_DIR:
  all_results.json
  summary_table.csv

Files saved to FIG_DIR:
  ablation_summary.png
  all_models_per_class_accuracy.png
  comparison_summary.png
  efficientnet_b0_best_confusion_matrix.png
  efficientnet_b0_img128_ep10_lr0.0001_adam_augFalse_curves.png
  efficientnet_b0_img128_ep10_lr0.0001_adam_augTrue_curves.png
  efficientnet_b0_img128_ep10_lr0.0001_adamw_augFalse_curves.png
  efficientnet_b0_img128_ep10_lr0.0001_adamw_augTrue_curves.png
  efficientnet_b0_img128_ep10_lr0.0001_sgd_augFalse_curves.png
  efficientnet_b0_img128_ep10_lr0.0001_sgd_augTrue_curves.png
  efficientnet_b0_img128_ep10_lr0.001_adam_augFalse_curves.png
  efficientnet_b0_img128_ep10_lr0.001_adam_augTrue_curves.png
  efficientnet_b0_img128_ep10_lr0.001_adamw_augFalse_curves.png
  efficientnet_b0_img128_ep10_lr0.001_adamw_augTrue_curves.png
  efficientnet_b0_img128_ep10_lr0.001_sgd_augFalse_curves.png
  efficientnet_b0_img128_ep10_lr0.001_sgd_augTrue_curves.png
  efficientnet_b0_img128_ep5_lr0.0001_adam_augFalse_curves.png
  efficientnet_b0_img128_ep5_lr0.0001_adam_augTrue_curves.png
  efficientnet_b0_img128_ep5_lr0.0001_adamw_augFalse_curves.png
  efficientnet_b0_img128_ep5_lr0.0001_adamw_augTrue_curves.png
  efficientnet_b0_img128_ep5_lr0.0001_sgd_augFalse_curves.png
  efficientnet_b0_img128_ep5_lr0.0001_sgd_augTrue_curves.png
  efficientnet_b0_img128_ep5_lr0.001_adam_augFalse_curves.png
  efficientnet_b0_img128_ep5_lr0.001_adam_augTrue_curves.png
  efficientnet_b0_img128_ep5_lr0.001_adamw_augFalse_curves.png
  efficientnet_b0_img128_ep5_lr0.001_adamw_augTrue_curves.png
  efficientnet_b0_img128_ep5_lr0.001_sgd_augFalse_curves.png
  efficientnet_b0_img128_ep5_lr0.001_sgd_augTrue_curves.png
  efficientnet_b0_img224_ep10_lr0.0001_adam_augFalse_curves.png
  efficientnet_b0_img224_ep10_lr0.0001_adam_augTrue_curves.png
  efficientnet_b0_img224_ep10_lr0.0001_adamw_augFalse_curves.png
  efficientnet_b0_img224_ep10_lr0.0001_adamw_augTrue_curves.png
  efficientnet_b0_img224_ep10_lr0.0001_sgd_augFalse_curves.png
  efficientnet_b0_img224_ep10_lr0.0001_sgd_augTrue_curves.png
  efficientnet_b0_img224_ep10_lr0.001_adam_augFalse_curves.png
  efficientnet_b0_img224_ep10_lr0.001_adam_augTrue_curves.png
  efficientnet_b0_img224_ep10_lr0.001_adamw_augFalse_curves.png
  efficientnet_b0_img224_ep10_lr0.001_adamw_augTrue_curves.png
  efficientnet_b0_img224_ep10_lr0.001_sgd_augFalse_curves.png
  efficientnet_b0_img224_ep10_lr0.001_sgd_augTrue_curves.png
  efficientnet_b0_img224_ep5_lr0.0001_adam_augFalse_curves.png
  efficientnet_b0_img224_ep5_lr0.0001_adam_augTrue_curves.png
  efficientnet_b0_img224_ep5_lr0.0001_adamw_augFalse_curves.png
  efficientnet_b0_img224_ep5_lr0.0001_adamw_augTrue_curves.png
  efficientnet_b0_img224_ep5_lr0.0001_sgd_augFalse_curves.png
  efficientnet_b0_img224_ep5_lr0.0001_sgd_augTrue_curves.png
  efficientnet_b0_img224_ep5_lr0.001_adam_augFalse_curves.png
  efficientnet_b0_img224_ep5_lr0.001_adam_augTrue_curves.png
  efficientnet_b0_img224_ep5_lr0.001_adamw_augFalse_curves.png
  efficientnet_b0_img224_ep5_lr0.001_adamw_augTrue_curves.png
  efficientnet_b0_img224_ep5_lr0.001_sgd_augFalse_curves.png
```

```
efficientnet_b0_img224_ep5_lr0.001_sgd_augTrue_curves.png
resnet18_best_confusion_matrix.png
resnet18_img128_ep10_lr0.0001_adam_augFalse_curves.png
resnet18_img128_ep10_lr0.0001_adam_augTrue_curves.png
resnet18_img128_ep10_lr0.0001_adamw_augFalse_curves.png
resnet18_img128_ep10_lr0.0001_adamw_augTrue_curves.png
resnet18_img128_ep10_lr0.0001_sgd_augFalse_curves.png
resnet18_img128_ep10_lr0.0001_sgd_augTrue_curves.png
resnet18_img128_ep10_lr0.001_adam_augFalse_curves.png
resnet18_img128_ep10_lr0.001_adam_augTrue_curves.png
resnet18_img128_ep10_lr0.001_adamw_augFalse_curves.png
resnet18_img128_ep10_lr0.001_adamw_augTrue_curves.png
resnet18_img128_ep10_lr0.001_sgd_augFalse_curves.png
resnet18_img128_ep10_lr0.001_sgd_augTrue_curves.png
resnet18_img128_ep5_lr0.0001_adam_augFalse_curves.png
resnet18_img128_ep5_lr0.0001_adam_augTrue_curves.png
resnet18_img128_ep5_lr0.0001_adamw_augFalse_curves.png
resnet18_img128_ep5_lr0.0001_adamw_augTrue_curves.png
resnet18_img128_ep5_lr0.0001_sgd_augFalse_curves.png
resnet18_img128_ep5_lr0.0001_sgd_augTrue_curves.png
resnet18_img128_ep5_lr0.001_adam_augFalse_curves.png
resnet18_img128_ep5_lr0.001_adam_augTrue_curves.png
resnet18_img128_ep5_lr0.001_adamw_augFalse_curves.png
resnet18_img128_ep5_lr0.001_adamw_augTrue_curves.png
resnet18_img128_ep5_lr0.001_sgd_augFalse_curves.png
resnet18_img128_ep5_lr0.001_sgd_augTrue_curves.png
resnet18_img224_ep10_lr0.0001_adam_augFalse_curves.png
resnet18_img224_ep10_lr0.0001_adam_augTrue_curves.png
resnet18_img224_ep10_lr0.0001_adamw_augFalse_curves.png
resnet18_img224_ep10_lr0.0001_adamw_augTrue_curves.png
resnet18_img224_ep10_lr0.0001_sgd_augFalse_curves.png
resnet18_img224_ep10_lr0.0001_sgd_augTrue_curves.png
resnet18_img224_ep10_lr0.001_adam_augFalse_curves.png
resnet18_img224_ep10_lr0.001_adam_augTrue_curves.png
resnet18_img224_ep10_lr0.001_adamw_augFalse_curves.png
resnet18_img224_ep10_lr0.001_adamw_augTrue_curves.png
resnet18_img224_ep10_lr0.001_sgd_augFalse_curves.png
resnet18_img224_ep10_lr0.001_sgd_augTrue_curves.png
resnet18_img224_ep5_lr0.0001_adam_augFalse_curves.png
resnet18_img224_ep5_lr0.0001_adam_augTrue_curves.png
resnet18_img224_ep5_lr0.0001_adamw_augFalse_curves.png
resnet18_img224_ep5_lr0.0001_adamw_augTrue_curves.png
resnet18_img224_ep5_lr0.0001_sgd_augFalse_curves.png
resnet18_img224_ep5_lr0.0001_sgd_augTrue_curves.png
resnet18_img224_ep5_lr0.001_adam_augFalse_curves.png
resnet18_img224_ep5_lr0.001_adam_augTrue_curves.png
resnet18_img224_ep5_lr0.001_adamw_augFalse_curves.png
resnet18_img224_ep5_lr0.001_adamw_augTrue_curves.png
resnet18_img224_ep5_lr0.001_sgd_augFalse_curves.png
resnet18_img224_ep5_lr0.001_sgd_augTrue_curves.png
svm_best_confusion_matrix.png
vit_small_best_confusion_matrix.png
vit_small_img224_ep10_lr0.0001_adam_augFalse_curves.png
vit_small_img224_ep10_lr0.0001_adam_augTrue_curves.png
vit_small_img224_ep10_lr0.0001_adamw_augFalse_curves.png
vit_small_img224_ep10_lr0.0001_adamw_augTrue_curves.png
```

```
  vit_small_img224_ep10_lr0.0001_sgd_augFalse_curves.png
  vit_small_img224_ep10_lr0.0001_sgd_augTrue_curves.png
  vit_small_img224_ep10_lr0.001_adam_augFalse_curves.png
  vit_small_img224_ep10_lr0.001_adam_augTrue_curves.png
  vit_small_img224_ep10_lr0.001_adamw_augFalse_curves.png
  vit_small_img224_ep10_lr0.001_adamw_augTrue_curves.png
  vit_small_img224_ep10_lr0.001_sgd_augFalse_curves.png
  vit_small_img224_ep10_lr0.001_sgd_augTrue_curves.png
  vit_small_img224_ep5_lr0.0001_adam_augFalse_curves.png
  vit_small_img224_ep5_lr0.0001_adam_augTrue_curves.png
  vit_small_img224_ep5_lr0.0001_adamw_augFalse_curves.png
  vit_small_img224_ep5_lr0.0001_adamw_augTrue_curves.png
  vit_small_img224_ep5_lr0.0001_sgd_augFalse_curves.png
  vit_small_img224_ep5_lr0.0001_sgd_augTrue_curves.png
  vit_small_img224_ep5_lr0.001_adam_augFalse_curves.png
  vit_small_img224_ep5_lr0.001_adam_augTrue_curves.png
  vit_small_img224_ep5_lr0.001_adamw_augFalse_curves.png
  vit_small_img224_ep5_lr0.001_adamw_augTrue_curves.png
  vit_small_img224_ep5_lr0.001_sgd_augFalse_curves.png
  vit_small_img224_ep5_lr0.001_sgd_augTrue_curves.png

Full experiment grid complete ✓
Best overall model: vit_small_img224_ep10_lr0.001_adamw_augFalse
Best accuracy     : 98.23%
```